

k8s 入门到微服务项目实战



▼ ① I. 课程简介

▼ 背景介绍

- 市场需求
- 技术竞争力
- 行业发展方向

▼ 课程解读

▼ 面向人群

- 运维工程师
- Java 开发
- 架构师
- 技术经理

▼ 前置学习

▼ 必须

- Linux 基础命令
- Docker

▼ 可选

- Java 微服务开发
- Redis
- Elasticsearch
- Prometheus
- Jenkins

▪ 模块解读

▼ 课程目标

- 深入理解 k8s 各大资源对象以及最佳实践
- 熟练运用 k8s 各项调度策略
- 掌握 k8s 网络原理及应用
- 熟练掌握 pod 控制器及运用场景
- 熟练掌握 k8s 微服务 DevOps 实战

▼ ② II. 核心概念篇

▼ 认识 Kubernetes

- 什么是 Kubernetes?

Kubernetes 是一个开源的，用于管理云平台中多个主机上的容器化的应用，Kubernetes 的目标是让部署容器化的应用简单并且高效（powerful），Kubernetes 提供了应用部署，规划，更新，维护的一种机制。

Kubernetes 这个名字源于希腊语，意为“舵手”或“飞行员”。k8s 这个缩写是因为 k 和 s 之间有八个字符的关系。Google 在 2014 年开源了 Kubernetes 项目。Kubernetes 建立在 [Google 大规模运行生产工作负载十几年经验](#)的基础上，结合了社区中最优秀的想法和实践。

▼ 为什么需要 Kubernetes?

▼ 应用部署的三大阶段

- 传统部署

程序员/运维工程师手动操作部署应用，直接将应用部署在目标机器上，由于资源不隔离，容易出现资源争抢、依赖冲突等各方面问题。

- 虚拟化部署

利用 OpenStack / VMware 等虚拟化技术，将一台目标机器虚拟化为多个虚拟机，按照需求将应用部署到不同的虚拟机中，对虚拟机进行动态的水平扩容等管理操作。

相对传统部署自动化、资源隔离的能力提升了，带来的问题是虚拟化的逻辑过重，导致效率不高，且耗费资源较多。

- 容器化部署

可以理解为轻量级的虚拟化，完美弥补虚拟化技术过重的问题，且由于直接共享主机硬件资源，只是通过系统提供的命名空间等技术实现资源隔离，损耗更小，且效率更高。

▼ k8s 的特点

- 自我修复
- 弹性伸缩
- 自动部署和回滚
- 服务发现和负载均衡
- 机密和配置管理
- 存储编排
- 批处理

▼ 企业级容器调度平台

▼ Apache Mesos

▪ 基本概念

Mesos 是一个分布式调度系统内核，早于 Docker 产生，Mesos 作为资源管理器，从 DC/OS (数据中心操作系统)的角度提供资源视图。主/从结构工作模式，主节点分配任务，并用从节点上的 Executor 负责执行，通过 Zookeeper 给主节点提供服务注册、服务发现功能。通过 Framework Marathon 提供容器调度的能力。

▪ 优势

经过时间的检验，作为资源管理器的 Apache Mesos 在容器之前就已经出现很久了，支持运行容器化化和非容器化的工作负载。可以支持应用程序的健康检查，开放的架构。支持多个框架和多个调度器，通过不同的 Framework 可以运行 Hadoop/Spark/MPI等多种不同的任务。

支持超大型规模的节点管理，模拟测试支持超过 5w+ 节点，在大规模上拥有较大优势。

▼ Docker Swarm

▪ 基本概念

Docker Swarm 是一个由 Docker 开发的调度框架。由 Docker 自身开发的好处之一就是标准 Docker API 的使用，Swarm 由多个代理 (Agent) 组成，把这些代理称之为节点 (Node)。这些节点就是主机，这些主机在启动 Docker Daemon 的时候就会打开相应的端口，以此支持 Docker 远程 API。这些机器会根据 Swarm 调度器分配给它们的任务，拉取和运行不同的镜像。

▪ 优势

从 Docker 1.12 版本开始，Swarm 随 Docker 一起默认安装发布。由于随 Docker 引擎一起发布，无需额外安装，配置简单。支持服务注册、服务发现，内置 Overlay Network 以及 Load Balancer。与 Docker CLI 非常类似的操作命令，对熟悉 Docker 的人非常容易上手学习。

入门门槛、学习成本较低，使用更便捷，适用于中小型系统。

▼ Google Kubernetes

▪ 基本概念

Kubernetes 是基于 Google 在过去十五年来大量生产环境中运行工作负载的经验。Kubernetes 的实现参考了 Google 内部的资源调度框架，但并不是 Borg 的内部容器编排系统的开源，而是借鉴 Google 从运行 Borg 获得的经验教训，形成了 Kubernetes 项目。

它使用 Label 和 Pod 的概念来将容器划分为逻辑单元。Pods 是同地协作（co-located）容器的集合，这些容器被共同部署和调度，形成了一个服务，这是 Kubernetes 和其他两个框架的主要区别。相比于基于相似度的容器调度方式（就像 Swarm 和 Mesos），这个方法简化了对集群的管理。

▪ 优势

最流行等容器编排解决方案框架，基于 Google 庞大的生态圈及社区产生的产品。通过 Pods 这一抽象的概念，解决 Container 之间的依赖于通信问题。Pods, Services, Deployments 是独立部署的部分，可以通过 Selector 提供更多的灵活性。内置服务注册表和负载均衡。

适用度更广，功能更强大，相较于 Mesos 来说节点规模较小，

▼ 集群架构与组件

▼ 相关组件

▼ 控制面板组件 (Master)

▪ kube-apiserver

API 服务器是 Kubernetes [控制平面](#)的组件，该组件负责公开了 Kubernetes API，负责处理接受请求的工作。API 服务器是 Kubernetes 控制平面的前端。

Kubernetes API 服务器的主要实现是 [kube-apiserver](#)。kube-apiserver 设计上考虑了水平扩缩，也就是说，它可通过部署多个实例来进行扩缩。你可以运行 kube-apiserver 的多个实例，并在这些实例之间平衡流量。

- **kube-controller-manager**

kube-controller-manager 是**控制平面**的组件，负责运行**控制器**进程。

从逻辑上讲，每个**控制器**都是一个单独的进程，但是为了降低复杂性，它们都被编译到同一个可执行文件，并在同一个进程中运行。

这些控制器包括：

- 节点控制器（Node Controller）：负责在节点出现故障时进行通知和响应
- 任务控制器（Job Controller）：监测代表一次性任务的 Job 对象，然后创建 Pods 来运行这些任务直至完成
- 端点分片控制器（EndpointSlice controller）：填充端点分片（EndpointSlice）对象（以提供 Service 和 Pod 之间的链接）。
- 服务账号控制器（ServiceAccount controller）：为新的命名空间创建默认的服务账号（ServiceAccount）。

- **cloud-controller-manager**

嵌入了特定于云平台的控制逻辑。云控制器管理器（Cloud Controller Manager）允许你将你的集群连接到云提供商的 API 之上，并将与该云平台交互的组件同与你的集群交互的组件分离开来。

cloud-controller-manager 仅运行特定于云平台的控制器。因此如果你在自己的环境中运行 Kubernetes，或者在本地计算机中运行学习环境，所部署的集群不需要有云控制器管理器。

与 kube-controller-manager 类似，cloud-controller-manager 将若干逻辑上独立的控制回路组合到同一个可执行文件中，供你以同一进程的方式运行。你可以对其执行水平扩容（运行不止一个副本）以提升性能或者增强容错能力。

- **kube-scheduler**

scheduler 负责资源的调度，按照预定的调度策略将 Pod 调度到相应的机器上；

- etcd

一致且高度可用的键值存储，用作 Kubernetes 的所有集群数据的后台数据库。

如果你的 Kubernetes 集群使用 etcd 作为其后台数据库，请确保你针对这些数据有一份 [备份](#) 计划。

你可以在官方[文档](#)中找到有关 etcd 的深入知识。

早期数据存放在内存，现在已经是持久化存储的了。

- ▼ 节点组件

- kubelet

kubelet 负责维护容器的生命周期，同时也负责 Volume (CVI) 和网络 (CNI) 的管理；

- kube-proxy

kube-proxy 负责为 Service 提供 cluster 内部的服务发现和负载均衡；

- container runtime

Container runtime 负责镜像管理以及 Pod 和容器的真正运行 (CRI) ；

Kubernetes 支持许多容器运行环境，例如 [containerd](#)、[CRI-O](#) 以及 [Kubernetes CRI \(容器运行环境接口\)](#) 的其他任何实现。

- ▼ 附加组件

- kube-dns

kube-dns 负责为整个集群提供 DNS 服务

- Ingress Controller

Ingress Controller 为服务提供外网入口

- Prometheus

Prometheus 提供资源监控

- Dashboard

Dashboard 提供 GUI

- Federation

Federation 提供跨可用区的集群

- Fluentd-elasticsearch

Fluentd-elasticsearch 提供集群日志采集、存储与查询

▼ 分层架构

- 生态系统

在接口层之上的庞大容器集群管理调度的生态系统，可以划分为两个范畴：

- Kubernetes 外部：日志、监控、配置管理、CI、CD、Workflow、FaaS、OTS 应用、ChatOps 等
- Kubernetes 内部：CRI、CNI、CVI、镜像仓库、Cloud Provider、集群自身的配置和管理等

- 接口层

kubectl 命令行工具、客户端 SDK 以及集群联邦

- 管理层

系统度量（如基础设施、容器和网络的度量），自动化（如自动扩展、动态 Provision 等）以及策略管理（RBAC、Quota、PSP、NetworkPolicy 等）

- 应用层

部署（无状态应用、有状态应用、批处理任务、集群应用等）和路由（服务发现、DNS 解析等）

- 核心层

Kubernetes 最核心的功能，对外提供 API 构建高层的应用，对内提供插件式应用执行环境

▼ 核心概念与专业术语

▼ 服务的分类

▼ 无状态

需要在服务运行环境下存储数据的服务，我们称之为我

▼ 代表应用

- Nginx
- Apache

- 优点：对客户端透明，无依赖关系，可以高效实现扩容、迁移
- 缺点：不能存储数据，需要额外的数据服务支撑

▼ 有状态

▼ 代表应用

- MySQL
- Redis

- 优点：可以独立存储数据，实现数据管理
- 缺点：集群环境下需要实现主从、数据同步、备份、水平扩容复杂

▼ 资源和对象

Kubernetes 中的所有内容都被抽象为“资源”，如 Pod、Service、Node 等都是资源。“对象”就是“资源”的实例，是持久化的实体。如某个具体的 Pod、某个具体的 Node。Kubernetes 使用这些实体去表示整个集群的状态。

对象的创建、删除、修改都是通过“Kubernetes API”，也就是“Api Server”组件提供的 API 接口，这些是 RESTful 风格的 Api，与 k8s 的“万物皆对象”理念相符。命令行工具“kubectl”，实际上也是调用 kubernetes api。

K8s 中的资源类别有很多种，kubectl 可以通过配置文件来创建这些“对象”，配置文件更像是描述对象“属性”的文件，配置文件格式可以是“JSON”或“YAML”，常用“YAML”。

▼ 资源的分类

▼ 元数据类型

▪ Horizontal Pod Autoscaler (HPA)

Pod 自动扩容：可以根据 CPU 使用率或自定义指标 (metrics) 自动对 Pod 进行扩/缩容。

- 控制管理器每隔30s（可以通过--horizontal-pod-autoscaler-sync-period修改）查询metrics的资源使用情况
- 支持三种metrics类型
 - 预定义metrics（比如Pod的CPU）以利用率的方式计算
 - 自定义的Pod metrics，以原始值（raw value）的方式计算
 - 自定义的object metrics
- 支持两种metrics查询方式：Heapster和自定义的REST API
- 支持多metrics

▪ PodTemplate

Pod Template 是关于 Pod 的定义，但是被包含在其他的 Kubernetes 对象中（例如 Deployment、StatefulSet、DaemonSet 等控制器）。控制器通过 Pod Template 信息来创建 Pod。

▪ LimitRange

可以对集群内 Request 和 Limits 的配置做一个全局的统一的限制，相当于批量设置了某一个范围内（某个命名空间）的 Pod 的资源使用限制。

▼ 集群级

▪ Namespace

Kubernetes 支持多个虚拟集群，它们底层依赖于同一个物理集群，这些虚拟集群被称为命名空间。

作用是用于实现多团队/环境的资源隔离。

命名空间 namespace 是 k8s 集群级别的资源，可以给不同的用户、租户、环境或项目创建对应的命名空间。

默认 namespace：

- kube-system 主要用于运行系统级资源，存放 k8s 自身的组件
- kube-public 此命名空间是自动创建的，并且可供所有用户（包括未经过身份验证的用户）读取。此命名空间主要用于集群使用，关联的一些资源在集群中是可见的并且可以公开读取。此命名空间的公共方面知识一个约定，但不是非要这么要求。
- default 未指定名称空间的资源就是 default，即你在创建pod 时如果没有指定 namespace，则会默认使用 default

▪ Node

不像其他的资源（如 Pod 和 Namespace），Node 本质上不是Kubernetes 来创建的，Kubernetes 只是管理 Node 上的资源。虽然可以通过 Manifest 创建一个Node对象（如下 json 所示），但 Kubernetes 也只是去检查是否真的是有这么一个 Node，如果检查失败，也不会往上调度 Pod。

▪ ClusterRole

ClusterRole 是一组权限的集合，但与 Role 不同的是，ClusterRole 可以在包括所有 Namespace 和集群级别的资源或非资源类型进行鉴权。

▪ ClusterRoleBinding

ClusterRoleBinding：将 Subject 绑定到 ClusterRole，ClusterRoleBinding 将使规则在所有命名空间中生效。

▼ 命名空间级

▼ 工作负载型

▼ Pod

Pod（容器组）是 Kubernetes 中最小的可部署单元。一个 Pod（容器组）包含了一个应用程序容器（某些情况下是多个容器）、存储资源、一个唯一的网络 IP 地址、以及一些确定容器该如何运行的选项。Pod 容器组代表了 Kubernetes 中一个独立的应用程序运行实例，该实例可能由单个容器或者几个紧耦合在一起的容器组成。

Docker 是 Kubernetes Pod 中使用最广泛的容器引擎；Kubernetes Pod 同时也支持其他类型的容器引擎。

Kubernetes 集群中的 Pod 存在如下两种使用途径：

- 一个 Pod 中只运行一个容器。“one-container-per-pod”是 Kubernetes 中最常见的使用方式。此时，您可以认为 Pod 容器组是该容器的 wrapper，Kubernetes 通过 Pod 管理容器，而不是直接管理容器。
- 一个 Pod 中运行多个需要互相协作的容器。您可以将多个紧密耦合、共享资源且始终在一起运行的容器编排在同一个 Pod 中，可能的情况有：

▪ 副本 (replicas)

先引入“副本”的概念——一个 Pod 可以被复制成多份，每一份可被称之为一个“副本”，这些“副本”除了一些描述性的信息（Pod 的名字、uid 等）不一样以外，其它信息都是一样的，譬如 Pod 内部的容器、容器数量、容器里面运行的应用等的这些信息都是一样的，这些副本提供同样的功能。

Pod 的“**控制器**”通常包含一个名为“replicas”的属性。“replicas”属性则指定了特定 Pod 的副本的数量，当当前集群中该 Pod 的数量与该属性指定的值不一致时，k8s 会采取一些策略去使得当前状态满足配置的要求。

▼ 控制器

当 Pod 被创建出来，Pod 会被调度到集群中的节点上运行，Pod 会在该节点上一直保持运行状态，直到进程终止、Pod 对象被删除、Pod 因节点资源不足而被驱逐或者节点失效为止。Pod 并不会自愈，当节点失效，或者调度 Pod 的这一操作失败了，Pod 就该被删除。如此，单单用 Pod 来部署应用，是不稳定不安全的。

Kubernetes 使用更高级的资源对象“**控制器**”来实现对 Pod 的管理。控制器可以为您创建和管理多个 Pod，管理副本和上线，并在集群范围内提供自修复能力。例如，如果一个节点失败，控制器可以在不同的节点上调度一样的替身来自动替换 Pod。

▼ 适用无状态服务

▪ ReplicationController (RC)

Replication Controller 简称 RC，RC 是 Kubernetes 系统中的核心概念之一，简单来说，RC 可以保证在任意时间运行 Pod 的副本数量，能够保证 Pod 总是可用的。如果实际 Pod 数量比指定的多那就结束掉多余的，如果实际数量比指定的少就新启动一些 Pod，当 Pod 失败、被删除或者挂掉后，RC 都会去自动创建新的 Pod 来保证副本数量，所以即使只有一个 Pod，我们也应该使用 RC 来管理我们的 Pod。可以说，通过 ReplicationController，Kubernetes 实现了 Pod 的高可用性。

▼ ReplicaSet (RS)

RC (ReplicationController) 主要的作用就是用来确保容器应用的副本数始终保持在用户定义的副本数。即如果有容器异常退出，会自动创建新的 Pod 来替代；而如果异常多出来的容器也会自动回收（已经成为过去时），在 v1.11 版本废弃。

Kubernetes 官方建议使用 RS (ReplicaSet) 替代 RC (ReplicationController) 进行部署，RS 跟 RC 没有本质的不同，只是名字不一样，并且 RS 支持集合式的 selector。

▪ Label 和 Selector

label (标签) 是附加到 Kubernetes 对象 (比如 Pods) 上的键值对，用于区分对象 (比如 Pod、Service)。label 旨在用于指定对用户有意义且相关的对象的标识属性，但不直接对核心系统有语义含义。label 可以用于组织和选择对象的子集。label 可以在创建时附加到对象，随后可以随时添加和修改。可以像 namespace 一样，使用 label 来获取某类对象，但 label 可以与 selector 一起配合使用，用表达式对条件加以限制，实现更精确、更灵活的资源查找。

label 与 selector 配合，可以实现对象的“关联”，“Pod 控制器”与 Pod 是相关联的——“Pod 控制器”依赖于 Pod，可以给 Pod 设置 label，然后给“控制器”设置对应的 selector，这就实现了对象的关联。

▼ Deployment

Deployment 为 Pod 和 Replica Set 提供声明式更新。

你只需要在 Deployment 中描述你想要的目标状态是什么，Deployment controller 就会帮你将 Pod 和 Replica Set 的实际状态改变到你的目标状态。你可以定义一个全新的 Deployment，也可以创建一个新的替换旧的 Deployment。

- 创建 Replica Set / Pod
- 滚动升级/回滚
- 平滑扩容和缩容
- 暂停与恢复 Deployment

▼ 适用有状态服务

▼ StatefulSet

StatefulSet 中每个 Pod 的 DNS 格式为 `statefulSetName-{0..N-1}.serviceName.namespace.svc.cluster.local`

- serviceName 为 Headless Service 的名字
- 0..N-1 为 Pod 所在的序号，从 0 开始到 N-1
- statefulSetName 为 StatefulSet 的名字
- namespace 为服务所在的 namespace，Headless Service 和 StatefulSet 必须在相同的 namespace
- .cluster.local 为 Cluster Domain

▼ 主要特点

- 稳定的持久化存储
即 Pod 重新调度后还是能访问到相同的持久化数据，基于 PVC 来实现
- 稳定的网络标志
稳定的网络标志，即 Pod 重新调度后其 PodName 和 HostName 不变，基于 Headless Service（即没有 Cluster IP 的 Service）来实现
- 有序部署，有序扩展
有序部署，有序扩展，即 Pod 是有顺序的，在部署或者扩展的时候要依据定义的顺序依次依次进行（即从 0 到 N-1，在下一个 Pod 运行之前所有之前的 Pod 必须都是 Running 和 Ready 状态），基于 init containers 来实现
- 有序收缩，有序删除
有序收缩，有序删除（即从 N-1 到 0）

▼ 组成

- Headless Service

用于定义网络标志 (DNS domain)

Domain Name Server: 域名服务

将域名与 ip 绑定映射关系

服务名 => 访问路径 (域名) => ip

- volumeClaimTemplate

用于创建 PersistentVolumes

- ▼ 注意事项

- kubernetes v1.5 版本以上才支持
- 所有Pod的Volume必须使用PersistentVolume或者是管理员事先创建好
- 为了保证数据安全, 删除StatefulSet时不会删除Volume
- StatefulSet 需要一个 Headless Service 来定义 DNS domain, 需要在 StatefulSet 之前创建好

- ▼ 守护进程

- DaemonSet

DemonSet 保证在每个 Node 上都运行一个容器副本, 常用来部署一些集群的日志、监控或者其他系统管理应用。典型的应用包括:

- 日志收集, 比如 fluentd, logstash 等
- 系统监控, 比如 Prometheus Node Exporter, collectd, New Relic agent, Ganglia gmond 等
- 系统程序, 比如 kube-proxy, kube-dns, glusterd, ceph 等

- ▼ 任务/定时任务

- Job

一次性任务, 运行完成后Pod销毁, 不再重新启动新容器。

- CronJob

CronJob 是在 Job 基础上加上了定时功能。

- ▼ 服务发现

- Service

“Service” 简写 “svc”。Pod 不能直接提供给外网访问，而是应该使用 service。Service 就是把 Pod 暴露出来提供服务，Service 才是真正的“服务”，它的中文名就叫“服务”。

可以说 Service 是一个应用服务的抽象，定义了 Pod 逻辑集合和访问这个 Pod 集合的策略。Service 代理 Pod 集合，对外表现为一个访问入口，访问该入口的请求将经过负载均衡，转发到后端 Pod 中的容器。

- Ingress

Ingress 可以提供外网访问 Service 的能力。可以把某个请求地址映射、路由到特定的 service。

ingress 需要配合 ingress controller 一起使用才能发挥作用，ingress 只是相当于路由规则的集合而已，真正实现路由功能的，是 Ingress Controller，ingress controller 和其它 k8s 组件一样，也是在 Pod 中运行。

- ▼ 存储

- Volume

数据卷，共享 Pod 中容器使用的数据。用来放持久化的数据，比如数据库数据。

- CSI

Container Storage Interface 是由来自 Kubernetes、Mesos、Docker 等社区成员联合制定的一个行业标准接口规范，旨在将任意存储系统暴露给容器化应用程序。

CSI 规范定义了存储提供商实现 CSI 兼容的 Volume Plugin 的最小操作集和部署建议。CSI 规范的主要焦点是声明 Volume Plugin 必须实现的接口。

- ▼ 特殊类型配置

- ConfigMap

用来放配置，与 Secret 是类似的，只是 ConfigMap 放的是明文的数据，Secret 是密文存放。

▪ Secret

Secret 解决了密码、token、密钥等敏感数据的配置问题，而不需要把这些敏感数据暴露到镜像或者 Pod Spec 中。Secret 可以以 Volume 或者环境变量的方式使用。

Secret 有三种类型：

- Service Account：用来访问 Kubernetes API，由 Kubernetes 自动创建，并且会自动挂载到 Pod 的 `/run/secrets/kubernetes.io/serviceaccount` 目录中；
- Opaque：base64 编码格式的 Secret，用来存储密码、密钥等；
- kubernetes.io/dockerconfigjson：用来存储私有 docker registry 的认证信息。

▪ DownwardAPI

downwardAPI 这个模式和其他模式不一样的地方在于它不是为了存放容器的数据也不是用来进行容器和宿主机的数据交换的，而是让 pod 里的容器能够直接获取到这个 pod 对象本身的一些信息。

downwardAPI 提供了两种方式用于将 pod 的信息注入到容器内部：

环境变量：用于单个变量，可以将 pod 信息和容器信息直接注入容器内部

volume 挂载：将 pod 信息生成为文件，直接挂载到容器内部中去

▼ 其他

▪ Role

Role 是一组权限的集合，例如 Role 可以包含列出 Pod 权限及列出 Deployment 权限，Role 用于给某个 Namespace 中的资源进行鉴权。

▪ RoleBinding

RoleBinding：将 Subject 绑定到 Role，RoleBinding 使规则在命名空间内生效。

▪ 资源清单

📄 k8s 的资源清单.md

创建 k8s 的对象都是通过 [yaml](#) 文件的形式进行配置的

▼ 对象规约和状态

对象是用来完成一些任务的，是持久的，是有目的性的，因此 kubernetes 创建一个对象后，将持续地工作以确保对象存在。当然，kubernetes 并不只是维持对象的存在这么简单，kubernetes 还管理着对象的方方面面。每个 Kubernetes 对象包含两个嵌套的对象字段，它们负责管理对象的配置，他们分别是 "spec" 和 "status"。

▪ 规约 (Spec)

"spec" 是 "规约"、"规格" 的意思，spec 是必需的，它描述了对对象的期望状态 (Desired State) —— 希望对象所具有的特征。当创建 Kubernetes 对象时，必须提供对象的规约，用来描述该对象的期望状态，以及关于对象的一些基本信息（例如名称）。

▪ 状态 (Status)

表示对象的实际状态，该属性由 k8s 自己维护，k8s 会通过一系列的控制器对对象进行管理，让对象尽可能的让实际状态与期望状态重合。

▪ 微服务项目 k8s 环境演示

▼ 3 III. 深入 k8s 实战进阶篇

▼ 搭建 Kubernetes 集群

▼ 搭建方案

▪ minikube

[minikube](#) 是一个工具，能让你在本地运行 Kubernetes。minikube 在你的个人计算机（包括 Windows、macOS 和 Linux PC）上运行一个一体化（all-in-one）或多节点的本地 Kubernetes 集群，以便你来尝试 Kubernetes 或者开展每天的开发工作。

[官方安装文档](#)

▼ kubeadm

你可以使用 [kubeadm](#) 工具来创建和管理 Kubernetes 集群。该工具能够执行必要的动作并用一种用户友好的方式启动一个可用的、安全的集群。

[安装 kubeadm](#) 展示了如何安装 kubeadm 的过程。一旦安装了 kubeadm，你就可以使用它来[创建一个集群](#)。

▼ 服务器要求

▼ 3 台服务器（虚拟机）

- k8s-master: 192.168.113.120
- k8s-node1: 192.168.113.121
- k8s-node2: 192.168.113.122

- 最低配置：2核、2G内存、20G硬盘

- 最好能联网，不能联网的话需要有提供对应镜像的私有仓库

▼ 软件环境

- 操作系统：CentOS 7
- Docker：20+
- k8s：1.23.6

▼ 安装步骤

▪ 1. 初始操作

```
# 关闭防火墙
systemctl stop firewalld
systemctl disable firewalld

# 关闭selinux
sed -i 's/enforcing/disabled/' /etc/selinux/config # 永久
setenforce 0 # 临时

# 关闭swap
swapoff -a # 临时
sed -ri 's/.*swap.*#&/' /etc/fstab # 永久

# 关闭完swap后，一定要重启一下虚拟机!!!
# 根据规划设置主机名
hostnamectl set-hostname <hostname>

# 在master添加hosts
cat >> /etc/hosts << EOF
192.168.113.120 k8s-master
192.168.113.121 k8s-node1
192.168.113.122 k8s-node2
EOF

# 将桥接的IPv4流量传递到iptables的链
cat > /etc/sysctl.d/k8s.conf << EOF
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
EOF

sysctl --system # 生效

# 时间同步
yum install ntpdate -y
ntpdate time.windows.com
```

▼ 2. 安装基础软件（所有节点）

▪ 2.1 安装 Docker

基于文档中的安装 Docker 文件进行安装

▪ 2.2 添加阿里云 yum 源

```
cat > /etc/yum.repos.d/kubernetes.repo << EOF
[kubernetes]
name=Kubernetes
baseurl=https://mirrors.aliyun.com/kubernetes/yum/repos/kubernetes-
el7-x86_64
enabled=1
gpgcheck=0
repo_gpgcheck=0

gpgkey=https://mirrors.aliyun.com/kubernetes/yum/doc/yum-key.gpg
https://mirrors.aliyun.com/kubernetes/yum/doc/rpm-package-key.gpg
EOF
```

▪ 2.3 安装 kubeadm、kubelet、kubectl

```
yum install -y kubelet-1.23.6 kubeadm-1.23.6 kubectl-1.23.6
```

```
systemctl enable kubelet
```

配置关闭 Docker 的 cgroups, 修改 /etc/docker/daemon.json, 加入以下内容

```
"exec-opts": ["native.cgroupdriver=systemd"]
```

重启 docker

```
systemctl daemon-reload
```

```
systemctl restart docker
```

▪ 3. 部署 Kubernetes Master

在 Master 节点下执行

```
kubeadm init \
  --apiserver-advertise-address=192.168.113.120 \
  --image-repository registry.aliyuncs.com/google_containers \
  --kubernetes-version v1.23.6 \
  --service-cidr=10.96.0.0/12 \
  --pod-network-cidr=10.244.0.0/16
```

安装成功后, 复制如下配置并执行

```
mkdir -p $HOME/.kube
```

```
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

```
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

```
kubectl get nodes
```

■ 4. 加入 Kubernetes Node

分别在 k8s-node1 和 k8s-node2 执行

下方命令可以在 k8s master 控制台初始化成功后复制 join 命令

```
kubeadm join 192.168.113.120:6443 --token w34ha2.66if2c8nwmeat9o7 --  
discovery-token-ca-cert-hash  
sha256:20e2227554f8883811c01edd850f0cf2f396589d32b57b9984de3353a  
7389477
```

如果初始化的 token 不小心清空了，可以通过如下命令获取或者重新申请

如果 token 已经过期，就重新申请

```
kubeadm token create
```

token 没有过期可以通过如下命令获取

```
kubeadm token list
```

获取 --discovery-token-ca-cert-hash 值，得到值后需要在前面拼接上 sha256:

```
openssl x509 -pubkey -in /etc/kubernetes/pki/ca.crt | openssl rsa -pubin -  
outform der 2>/dev/null | \  
openssl dgst -sha256 -hex | sed 's/^.* //'
```

■ 5. 部署 CNI 网络插件

在 master 节点上执行

下载 calico 配置文件，可能会网络超时

```
curl https://docs.projectcalico.org/manifests/calico.yaml -O
```

修改 calico.yaml 文件中的 CALICO_IPV4POOL_CIDR 配置，修改为与初始化的 cidr 相同

修改 IP_AUTODETECTION_METHOD 下的网卡名称

删除镜像 docker.io/ 前缀，避免下载过慢导致失败

```
sed -i 's#docker.io/##g' calico.yaml
```

- 7. 测试 kubernetes 集群

- # 创建部署

- kubectl create deployment nginx --image=nginx

- # 暴露端口

- kubectl expose deployment nginx --port=80 --type=NodePort

- # 查看 pod 以及服务信息

- kubectl get pod,svc

- 二进制安装

- 利用 k8s 官方 github 仓库下载二进制包安装，安装过程较复杂，但相对较为稳定，推荐生产环境使用。

- 命令行工具

- ▼ 命令行工具 kubectl

命令行工具 (kubectl)

Kubernetes 提供 kubectl 是使用 Kubernetes API 与 Kubernetes 集群的[控制面](#)进行通信的命令行工具。

这个工具叫做 kubectl。

更多命令

- 在任意节点使用 kubectl

- # 1. 将 master 节点中 /etc/kubernetes/admin.conf 拷贝到需要运行的服务器的 /etc/kubernetes 目录中

- scp /etc/kubernetes/admin.conf root@k8s-node1:/etc/kubernetes

- # 2. 在对应的服务器上配置环境变量

- echo "export KUBECONFIG=/etc/kubernetes/admin.conf" >> ~/.bash_profile

- source ~/.bash_profile

- ▼ 资源操作

■ 创建对象

```
$ kubectl create -f ./my-manifest.yaml # 创建资源
$ kubectl create -f ./my1.yaml -f ./my2.yaml # 使用多个文件创建资源
$ kubectl create -f ./dir # 使用目录下的所有清单文件来创建资源
$ kubectl create -f https://git.io/vPieo # 使用 url 来创建资源
$ kubectl run nginx --image=nginx # 启动一个 nginx 实例
$ kubectl explain pods,svc # 获取 pod 和 svc 的文档

# 从 stdin 输入中创建多个 YAML 对象
$ cat <<EOF | kubectl create -f -
apiVersion: v1
kind: Pod
metadata:
  name: busybox-sleep
spec:
  containers:
  - name: busybox
    image: busybox
    args:
    - sleep
    - "1000000"
---
apiVersion: v1
kind: Pod
metadata:
  name: busybox-sleep-less
spec:
  containers:
  - name: busybox
    image: busybox
    args:
    - sleep
    - "1000"
EOF

# 创建包含几个 key 的 Secret
$ cat <<EOF | kubectl create -f -
apiVersion: v1
kind: Secret
metadata:
  name: mysecret
type: Opaque
data:
  password: $(echo "s33msi4" | base64)
  username: $(echo "jane" | base64)
EOF
```

■ 显示和查找资源

Get commands with basic output

\$ kubectl get services	# 列出所有 namespace 中的所有 service
\$ kubectl get pods --all-namespaces	# 列出所有 namespace 中的所有 pod
\$ kubectl get pods -o wide	# 列出所有 pod 并显示详细信息
\$ kubectl get deployment my-dep	# 列出指定 deployment
\$ kubectl get pods --include-uninitialized	# 列出该 namespace 中的所有 pod 包括未初始化

使用详细输出来描述命令

```
$ kubectl describe nodes my-node
$ kubectl describe pods my-pod
```

```
$ kubectl get services --sort-by=.metadata.name # List Services Sorted by Name
```

根据重启次数排序列出 pod

```
$ kubectl get pods --sort-by='.status.containerStatuses[0].restartCount'
```

获取所有具有 app=cassandra 的 pod 中的 version 标签

```
$ kubectl get pods --selector=app=cassandra rc -o \
  jsonpath='{.items[*].metadata.labels.version}'
```

获取所有节点的 ExternalIP

```
$ kubectl get nodes -o jsonpath='{.items[*].status.addresses[?(@.type=="ExternalIP")].add
```

列出属于某个 PC 的 Pod 的名字

"jq" 命令用于转换复杂的 jsonpath, 参考 <https://stedolan.github.io/jq/>

```
$ sel=${$(kubectl get rc my-rc --output=json | jq -j '.spec.selector | to_entries | .[] |
$ echo ${kubectl get pods --selector=$sel --output=jsonpath={.items..metadata.name})}
```

查看哪些节点已就绪

```
$ JSONPATH='{range .items[*]}{@.metadata.name}:{range @.status.conditions[*]}{@.type}={@.
&& kubectl get nodes -o jsonpath="$JSONPATH" | grep "Ready=True"
```

列出当前 Pod 中使用的 Secret

```
$ kubectl get pods -o json | jq '.items[].spec.containers[].env[]?.valueFrom.secretKeyRef
```

<

>

■ 更新资源

```
$ kubectl rolling-update frontend-v1 -f frontend-v2.json # 滚动更新 pod frontend-
$ kubectl rolling-update frontend-v1 frontend-v2 --image=image:v2 # 更新资源名称并更新镜像
$ kubectl rolling-update frontend --image=image:v2 # 更新 frontend pod 中的
$ kubectl rolling-update frontend-v1 frontend-v2 --rollback # 退出已存在的进行中的滚
$ cat pod.json | kubectl replace -f - # 基于 stdin 输入的 JSON

# 强制替换, 删除后重新创建资源。会导致服务中断。
$ kubectl replace --force -f ./pod.json

# 为 nginx RC 创建服务, 启用本地 80 端口连接到容器上的 8000 端口
$ kubectl expose rc nginx --port=80 --target-port=8000

# 更新单容器 pod 的镜像版本 (tag) 到 v4
$ kubectl get pod mypod -o yaml | sed 's/\(image: myimage\):.*$/\1:v4/' | kubectl replace

$ kubectl label pods my-pod new-label=awesome # 添加标签
$ kubectl annotate pods my-pod icon-url=http://goo.gl/XXBTWq # 添加注解
$ kubectl autoscale deployment foo --min=2 --max=10 # 自动扩展 deployment
```

<  >

■ 修补资源

```
$ kubectl patch node k8s-node-1 -p '{"spec":{"unschedulable":true}}' # 部分更新节点

# 更新容器镜像; spec.containers[*].name 是必须的, 因为这是合并的关键字
$ kubectl patch pod valid-pod -p '{"spec":{"containers":[{"name":"kubernetes-serve-hostna

# 使用具有位置数组的 json 补丁更新容器镜像
$ kubectl patch pod valid-pod --type='json' -p='[{"op": "replace", "path": "/spec/contain

# 使用具有位置数组的 json 补丁禁用 deployment 的 livenessProbe
$ kubectl patch deployment valid-deployment --type json -p='[{"op": "remove", "path":
```

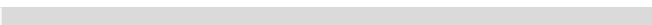
<  >

■ 编辑资源

```
$ kubectl edit svc/docker-registry # 编辑名为 docker-registry 的 service
$ KUBE_EDITOR="nano" kubectl edit svc/docker-registry # 使用其它编辑器
```

■ scale 资源

```
$ kubectl scale --replicas=3 rs/foo # Scale a replicaset nam
$ kubectl scale --replicas=3 -f foo.yaml # Scale a resource speci
$ kubectl scale --current-replicas=2 --replicas=3 deployment/mysql # If the deployment nan
$ kubectl scale --replicas=5 rc/foo rc/bar rc/baz # Scale multiple replicat
```

<  >

删除资源

```
$ kubectl delete -f ./pod.json # 删除 pod.json
$ kubectl delete pod,service baz foo # 删除名为 "baz" 的 pod 和服务
$ kubectl delete pods,services -l name=myLabel # 删除具有 name=myLabel 的 pod 和服务
$ kubectl delete pods,services -l name=myLabel --include-uninitialized # 删除具有 name=myLabel 的 pod 和服务，包括未初始化的资源
$ kubectl -n my-ns delete po,svc --all # 删除 my-ns 中的所有 pod 和服务
```

Pod 与集群

与运行的 Pod 交互

```
$ kubectl logs my-pod # dump 输出 pod 的日志 (stdout)
$ kubectl logs my-pod -c my-container # dump 输出 pod 中容器的日志 (stdout,
$ kubectl logs -f my-pod # 流式输出 pod 的日志 (stdout)
$ kubectl logs -f my-pod -c my-container # 流式输出 pod 中容器的日志 (stdout, p
$ kubectl run -i --tty busybox --image=busybox -- sh # 交互式 shell 的方式运行 pod
$ kubectl attach my-pod -i # 连接到运行中的容器
$ kubectl port-forward my-pod 5000:6000 # 转发 pod 中的 6000 端口到本地的 5000
$ kubectl exec my-pod -- ls / # 在已存在的容器中执行命令 (只有一个容器)
$ kubectl exec my-pod -c my-container -- ls / # 在已存在的容器中执行命令 (pod 中有多
$ kubectl top pod POD_NAME --containers # 显示指定 pod 和容器的指标度量
```

与节点和集群交互

```
$ kubectl cordon my-node # 标记 my-node 不可调度
$ kubectl drain my-node # 清空 my-node 以进行维护
$ kubectl uncordon my-node # 标记 my-node 可调度
$ kubectl top node my-node # 显示 my-node 的指标度量
$ kubectl cluster-info # 显示 master 和服务端点的信息
$ kubectl cluster-info dump # 将当前集群状态输出到文件
$ kubectl cluster-info dump --output-directory=/path/to/cluster-state # 将当前集群状态输出到指定目录
```

```
# 如果该键和影响的污点 (taint) 已存在，则使用指定的值替换
$ kubectl taint nodes foo dedicated=special-user:NoSchedule
```

资源类型与别名

📄 kubectl资源类型与别名.md

▼ pods

▪ po

▼ deployments

▪ deploy

▼ services

- svc
- ▼ namespace
 - ns
- ▼ nodes
 - no
- ▼ 格式化输出
 - ▼ 输出 json 格式
 - -o json
 - ▼ 仅打印资源名称
 - -o name
 - ▼ 以纯文本格式输出所有信息
 - -o wide
 - ▼ 输出 yaml 格式
 - -o yaml

▼ API 概述

官网文档: <https://kubernetes.io/zh-cn/docs/reference/using-api/>

REST API 是 Kubernetes 系统的重要部分，组件之间的所有操作和通信均由 API Server 处理的 REST API 调用，大多数情况下，API 定义和实现都符合标准的 HTTP REST 格式，可以通过 [kubect](#)l 命令管理工具或其他命令行工具来执行。

▼ 类型

- Alpha
 - 包含 alpha 名称的版本（例如v1alpha1）。
 - 该软件可能包含错误。启用一个功能可能会导致 bug。默认情况下，功能可能会被禁用。
 - 随时可能会丢弃对该功能的支持，恕不另行通知。
 - API 可能在以后的软件版本中以不兼容的方式更改，恕不另行通知。
 - 该软件建议仅在短期测试集群中使用，因为错误的风险增加和缺乏长期支持。

- Beta

- 包含 **beta** 名称的版本（例如 **v2beta3**）。
- 该软件经过很好的测试。启用功能被认为是安全的。默认情况下功能是开启的。
- 细节可能会改变，但功能在后续版本不会被删除
- 对象的模式或语义在随后的 beta 版本或 Stable 版本中可能以不兼容的方式发生变化。如果这种情况发生时，官方会提供迁移操作指南。这可能需要删除、编辑和重新创建API对象。
- 该版本在后续可能会更改一些不兼容地方，所以建议用于非关键业务，如果你有多个可以独立升级的集群，你也可以放宽此限制。
- **大家使用过的 Beta 版本后，可以多给社区反馈，如果此版本在后续更新后将不会有太大变化。**

- Stable

- 该版本名称命名方式：**vX** 这里 **X** 是一个整数。
- Stable 版本的功能特性，将出现在后续发布的软件版本中。

- ▼ 访问控制

- 认证
- 授权

- 废弃 api 说明

<https://kubernetes.io/zh-cn/docs/reference/using-api/deprecation-guide/>

- ▼ 深入 pod

▪ Pod 配置文件

```
apiVersion: v1 # api 文档版本
kind: Pod # 资源对象类型，也可以配置为像Deployment、StatefulSet这一类的对象
metadata: # Pod 相关的元数据，用于描述 Pod 的数据
  name: nginx-demo # Pod 的名称
  labels: # 定义 Pod 的标签
    type: app # 自定义 label 标签，名字为 type，值为 app
    test: 1.0.0 # 自定义 label 标签，描述 Pod 版本号
  namespace: 'default' # 命名空间的配置
spec: # 期望 Pod 按照这里的描述进行创建
  containers: # 对于 Pod 中的容器描述
    - name: nginx # 容器的名称
      image: nginx:1.7.9 # 指定容器的镜像
      imagePullPolicy: IfNotPresent # 镜像拉取策略，指定如果本地有就用本地的，如果没有就拉取远程的
      command: # 指定容器启动时执行的命令
        - nginx
        - -g
        - 'daemon off;' # nginx -g 'daemon off;'
      workingDir: /usr/share/nginx/html # 定义容器启动后的工作目录
      ports:
        - name: http # 端口名称
          containerPort: 80 # 描述容器内要暴露什么端口
          protocol: TCP # 描述该端口是基于哪种协议通信的
      - env: # 环境变量
        name: JVM_OPTS # 环境变量名称
        value: '-Xms128m -Xmx128m' # 环境变量的值
  resources:
    requests: # 最少需要多少资源
      cpu: 100m # 限制 cpu 最少使用 0.1 个核心
      memory: 128Mi # 限制内存最少使用 128兆
    limits: # 最多可以用多少资源
      cpu: 200m # 限制 cpu 最多使用 0.2 个核心
      memory: 256Mi # 限制 最多使用 256兆
  restartPolicy: OnFailure # 重启策略，只有失败的情况才会重启
```

▼ 探针

容器内应用的监测机制，根据不同的探针来判断容器应用当前的状态

▼ 类型

- **StartupProbe**

k8s 1.16 版本新增的探针，用于判断应用程序是否已经启动了。

当配置了 startupProbe 后，会先禁用其他探针，直到 startupProbe 成功后，其他探针才会继续。

作用：由于有时候不能准确预估应用一定是多长时间启动成功，因此配置另外两种方式不方便配置初始化时长来检测，而配置了 statupProbe 后，只有在应用启动成功了，才会执行另外两种探针，可以更加方便的结合使用另外两种探针使用。

startupProbe:

httpGet:

path: /api/startup

port: 80

- **LivenessProbe**

用于探测容器中的应用是否运行，如果探测失败，kubelet 会根据配置的重启策略进行重启，若没有配置，默认就认为容器启动成功，不会执行重启策略。

livenessProbe:

failureThreshold: 5

httpGet:

path: /health

port: 8080

scheme: HTTP

initialDelaySeconds: 60

periodSeconds: 10

successThreshold: 1

timeoutSeconds: 5

- **ReadinessProbe**

用于探测容器内的程序是否健康，它的返回值如果返回 success，那么就认为该容器已经完全启动，并且该容器是可以接收外部流量的。

readinessProbe:

failureThreshold: 3 # 错误次数

httpGet:

path: /ready

port: 8181

scheme: HTTP

periodSeconds: 10 # 间隔时间

successThreshold: 1

timeoutSeconds: 1

▼ 探测方式

▪ ExecAction

在容器内部执行一个命令，如果返回值为 0，则任务容器时健康的。

livenessProbe:

exec:

command:

- cat

- /health

▪ TCPSocketAction

通过 tcp 连接监测容器内端口是否开放，如果开放则证明该容器健康

livenessProbe:

tcpSocket:

port: 80

▪ HTTPGetAction

生产环境用的较多的方式，发送 HTTP 请求到容器内的应用程序，如果接口返回的状态码在 200~400 之间，则认为容器健康。

livenessProbe:

failureThreshold: 5

httpGet:

path: /health

port: 8080

scheme: HTTP

httpHeaders:

- name: xxx

value: xxx

▪ 参数配置

initialDelaySeconds: 60 # 初始化时间

timeoutSeconds: 2 # 超时时间

periodSeconds: 5 # 监测间隔时间

successThreshold: 1 # 检查 1 次成功就表示成功

failureThreshold: 2 # 监测失败 2 次就表示失败

▼ 生命周期

lifecycle:

postStart: # 容器创建完成后执行的动作，不能保证该操作一定在容器的 command 之前执行，一般不使用

exec: # 可以是 exec / httpGet / tcpSocket

command:

- sh
- -C
- 'mkdir /data'

preStop: # 在容器停止前执行的动作

httpGet: # 发送一个 http 请求

path: /

port: 80

exec: # 执行一个命令

command:

- sh
- -C
- sleep 9

▼ Pod 退出流程

▼ 删除操作

- Endpoint 删除 pod 的 ip 地址

- Pod 变成 Terminating 状态

变为删除中的状态后，会给 pod 一个宽限期，让 pod 去执行一些清理或销毁操作。

配置参数：

作用与 pod 中的所有容器

terminationGracePeriodSeconds: 30

containers:

- xxx

- 执行 preStop 的指令

▼ PreStop 的应用

如果应用销毁操作耗时需要比较长，可以在 preStop 按照如下方式进行配置

preStop:

exec:

command:

- sh
- -C
- 'sleep 20; kill pgrep java'

但是需要注意，由于 k8s 默认给 pod 的停止宽限时间为 30s，如果我们停止操作会超过 30s 时，不要光设置 sleep 50，还要将 terminationGracePeriodSeconds: 30 也更新成更长的时间，否则 k8s 最多只会在这个时间的基础上再宽限几秒，不会真正等待 50s

- 注册中心下线
- 数据清理
- 数据销毁

▼ 资源调度

▼ Label 和 Selector

▼ 标签 (Label)

- 配置文件

在各类资源的 metadata.labels 中进行配置

▼ kubectl

- 临时创建 label

kubectl label po <资源名称> app=hello

- 修改已经存在的标签

kubectl label po <资源名称> app=hello2 --overwrite

- 查看 label

selector 按照 label 单值查找节点

kubectl get po -A -l app=hello

查看所有节点的 labels

kubectl get po --show-labels

▼ 选择器 (Selector)

- 配置文件

在各对象的配置 spec.selector 或其他可以写 selector 的属性中编写

- kubectl

- # 匹配单个值, 查找 app=hello 的 pod

- kubectl get po -A -l app=hello

- # 匹配多个值

- kubectl get po -A -l 'k8s-app in (metrics-server, kubernetes-dashboard)'

- 或

- # 查找 version!=1 and app=nginx 的 pod 信息

- kubectl get po -l version!=1,app=nginx

- # 不等值 + 语句

- kubectl get po -A -l version!=1,'app in (busybox, nginx)'

- ▼ Deployment

- ▼ 功能

- 创建

- 创建一个 deployment

- kubectl create deploy nginx-deploy --image=nginx:1.7.9

- 或执行

- kubectl create -f xxx.yaml --record

- record 会在 annotation 中记录当前命令创建或升级了资源, 后续可以查看做过哪些变动操作。

- 查看部署信息

- kubectl get deployments

- 查看 rs

- kubectl get rs

- 查看 pod 以及展示标签, 可以看到是关联的那个 rs

- kubectl get pods --show-labels

▼ 滚动更新

只有修改了 deployment 配置文件中的 template 中的属性后，才会触发更新操作

修改 nginx 版本号

```
kubectl set image deployment/nginx-deployment nginx=nginx:1.9.1
```

或者通过 `kubectl edit deployment/nginx-deployment` 进行修改

查看滚动更新的过程

```
kubectl rollout status deploy <deployment_name>
```

查看部署描述，最后展示发生的事件列表也可以看到滚动更新过程

```
kubectl describe deploy <deployment_name>
```

通过 `kubectl get deployments` 获取部署信息，UP-TO-DATE 表示已经有多少副本达到了配置中要求的数目

通过 `kubectl get rs` 可以看到增加了一个新的 rs

通过 `kubectl get pods` 可以看到所有 pod 关联的 rs 变成了新的

▪ 多个滚动更新并行

假设当前有 5 个 nginx:1.7.9 版本，你想将版本更新为 1.9.1，当更新成功第三个以后，你马上又将期望更新的版本改为 1.9.2，那么此时会立马删除之前的三个，并且立马开启更新 1.9.2 的任务

- 回滚

有时候你可能想回退一个Deployment，例如，当Deployment不稳定时，比如一直crash looping。

默认情况下，kubernetes会在系统中保存前两次的Deployment的rollout历史记录，以便你可以随时会退（你可以修改revision history limit来更改保存的revision数）。

案例：

更新 deployment 时参数不小心写错，如 nginx:1.9.1 写成了 nginx:1.9l
kubectl set image deployment/nginx-deploy nginx=nginx:1.9l

监控滚动升级状态，由于镜像名称错误，下载镜像失败，因此更新过程会卡住

kubectl rollout status deployments nginx-deploy

结束监听后，获取 rs 信息，我们可以看到新增的 rs 副本数是 2 个
kubectl get rs

通过 kubectl get pods 获取 pods 信息，我们可以看到关联到新的 rs 的 pod，状态处于 ImagePullBackOff 状态

为了修复这个问题，我们需要找到需要回退的 revision 进行回退
通过 kubectl rollout history deployment/nginx-deploy 可以获取 revision 的列表

通过 kubectl rollout history deployment/nginx-deploy --revision=2 可以查看详细信息

确认要回退的版本后，可以通过 kubectl rollout undo deployment/nginx-deploy 可以回退到上一个版本

也可以回退到指定的 revision

kubectl rollout undo deployment/nginx-deploy --to-revision=2

再次通过 kubectl get deployment 和 kubectl describe deployment 可以看到，我们的版本已经回退到对应的 revision 上了

可以通过设置 .spec.revisionHistoryLimit 来指定 deployment 保留多少 revision，如果设置为 0，则不允许 deployment 回退了。

- 扩容缩容

通过 `kube scale` 命令可以进行自动扩容/缩容，以及通过 `kube edit` 编辑 `replcas` 也可以实现扩容/缩容

扩容与缩容只是直接创建副本数，没有更新 `pod template` 因此不会创建新的 `rs`

- 暂停与恢复

由于每次对 `pod template` 中的信息发生修改后，都会触发更新 `deployment` 操作，那么此时如果频繁修改信息，就会产生多次更新，而实际上只需要执行最后一次更新即可，当出现此类情况时我们就可以暂停 `deployment` 的 `rollout`

通过 `kubectl rollout pause deployment <name>` 就可以实现暂停，直到你下次恢复后才会继续进行滚动更新

尝试对容器进行修改，然后查看是否发生更新操作了

```
kubectl set image deploy <name> nginx=nginx:1.17.9
```

```
kubectl get po
```

通过以上操作可以看到实际并没有发生修改，此时我们再次进行修改一些属性，如限制 `nginx` 容器的最大 `cpu` 为 0.2 核，最大内存为 128M，最小内存为 64M，最小 `cpu` 为 0.1 核

```
kubectl set resources deploy <deploy_name> -c <container_name> --  
limits=cpu=200m,memory=128Mi --requests=cpu100m,memory=64Mi
```

通过格式化输出 `kubectl get deploy <name> -oyaml`，可以看到配置确实发生了修改，再通过 `kubectl get po` 可以看到 `pod` 没有被更新

那么此时我们再恢复 `rollout`，通过命令 `kubectl rollout deploy <name>`

恢复后，我们再次查看 `rs` 和 `po` 信息，我们可以看到就开始进行滚动更新操作了

```
kubectl get rs
```

```
kubectl get po
```

■ 配置文件

```
apiVersion: apps/v1 # deployment api 版本
kind: Deployment # 资源类型为 deployment
metadata: # 元信息
  labels: # 标签
    app: nginx-deploy # 具体的 key: value 配置形式
  name: nginx-deploy # deployment 的名字
  namespace: default # 所在的命名空间
spec:
  replicas: 1 # 期望副本数
  revisionHistoryLimit: 10 # 进行滚动更新后, 保留的历史版本数
  selector: # 选择器, 用于找到匹配的 RS
    matchLabels: # 按照标签匹配
      app: nginx-deploy # 匹配的标签key/value
  strategy: # 更新策略
    rollingUpdate: # 滚动更新配置
      maxSurge: 25% # 进行滚动更新时, 更新的个数最多可以超过期望副本数的个数/比例
      maxUnavailable: 25% # 进行滚动更新时, 最大不可用比例更新比例, 表示在所有副本数中, 最多可以有
      type: RollingUpdate # 更新类型, 采用滚动更新
  template: # pod 模板
    metadata: # pod 的元信息
      labels: # pod 的标签
        app: nginx-deploy
    spec: # pod 期望信息
      containers: # pod 的容器
        - image: nginx:1.7.9 # 镜像
          imagePullPolicy: IfNotPresent # 拉取策略
          name: nginx # 容器名称
      restartPolicy: Always # 重启策略
      terminationGracePeriodSeconds: 30 # 删除操作最多宽限多长时间
```

▼ StatefulSet

▼ 功能

- 创建

```
kubectl create -f web.yaml
```

查看 service 和 statefulset => sts

```
kubectl get service nginx
```

```
kubectl get statefulset web
```

查看 PVC 信息

```
kubectl get pvc
```

查看创建的 pod, 这些 pod 是有序的

```
kubectl get pods -l app=nginx
```

查看这些 pod 的 dns

运行一个 pod, 基础镜像为 busybox 工具包, 利用里面的 nslookup 可以看到 dns 信息

```
kubectl run -i --tty --image busybox dns-test --restart=Never --rm /bin/sh
```

```
nslookup web-0.nginx
```

- 扩容缩容

扩容

```
$ kubectl scale statefulset web --replicas=5
```

缩容

```
$ kubectl patch statefulset web -p '{"spec":{"replicas":3}}'
```

- ▼ 镜像更新

镜像更新 (目前还不支持直接更新 image, 需要 patch 来间接实现)

```
kubectl patch sts web --type='json' -p='[{"op": "replace", "path":
```

```
"/spec/template/spec/containers/0/image", "value": "nginx:1.9.1"}]'
```

- ▼ RollingUpdate

StatefulSet 也可以采用滚动更新策略, 同样是修改 pod template 属性后会触发更新, 但是由于 pod 是有序的, 在 StatefulSet 中更新时是基于 pod 的顺序倒序更新的

- 灰度发布

利用滚动更新中的 partition 属性, 可以实现简易的灰度发布的效果

例如我们有 5 个 pod, 如果当前 partition 设置为 3, 那么此时滚动更新时, 只会更新那些 序号 >= 3 的 pod

利用该机制, 我们可以通过控制 partition 的值, 来决定只更新其中一部分 pod, 确认没有问题后再主键增大更新的 pod 数量, 最终实现全部 pod 更新

- OnDelete

只有在 pod 被删除时会进行更新操作

- 删除

删除 StatefulSet 和 Headless Service

级联删除: 删除 statefulset 时会同时删除 pods

kubectl delete statefulset web

非级联删除: 删除 statefulset 时不会删除 pods, 删除 sts 后, pods 就没人管了, 此时再删除 pod 不

kubectl delete sts web --cascade=false

<

>

删除 service

kubectl delete service nginx

- 删除 pvc

StatefulSet删除后PVC还会保留着, 数据不再使用的话也需要删除

\$ kubectl delete pvc www-web-0 www-web-1

■ 配置文件

```
---
apiVersion: v1
kind: Service
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  ports:
    - port: 80
      name: web
  clusterIP: None
  selector:
    app: nginx
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: "nginx"
  replicas: 2
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.7.9
          ports:
            - containerPort: 80
              name: web
          volumeMounts:
            - name: www
              mountPath: /usr/share/nginx/html
      volumeClaimTemplates:
        - metadata:
            name: www
            annotations:
              volume.alpha.kubernetes.io/storage-class: anything
          spec:
            accessModes: [ "ReadWriteOnce" ]
            resources:
              requests:
                storage: 1Gi
```

▼ DaemonSet

■ 配置文件

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd
spec:
  template:
    metadata:
      labels:
        app: logging
        id: fluentd
        name: fluentd
    spec:
      containers:
        - name: fluentd-es
          image: agilestacks/fluentd-elasticsearch:v1.3.0
          env:
            - name: FLUENTD_ARGS
              value: -qq
          volumeMounts:
            - name: containers
              mountPath: /var/lib/docker/containers
            - name: varlog
              mountPath: /varlog
      volumes:
        - hostPath:
            path: /var/lib/docker/containers
            name: containers
        - hostPath:
            path: /var/log
            name: varlog
```

▼ 指定 Node 节点

DaemonSet 会忽略 Node 的 unschedulable 状态，有两种方式来指定 Pod 只运行在指定的 Node 节点上：

- nodeSelector：只调度到匹配指定 label 的 Node 上
- nodeAffinity：功能更丰富的 Node 选择器，比如支持集合操作
- podAffinity：调度到满足条件的 Pod 所在的 Node 上

■ nodeSelector

先为 Node 打上标签

```
kubectl label nodes k8s-node1 svc_type=microsvc
```

然后再 daemonset 配置中设置 nodeSelector

```
spec:
  template:
    spec:
      nodeSelector:
        svc_type: microsvc
```

- nodeAffinity

nodeAffinity 目前支持两种：requiredDuringSchedulingIgnoredDuringExecution 和 preferredDuringSchedulingIgnoredDuringExecution，分别代表必须满足条件和优选条件。

比如下面的例子代表调度到包含标签 wolfcode.cn/framework-name 并且值为 spring 或 springboot 的 Node 上，并且优选还带有标签 another-node-label-key=another-node-label-value 的Node。

```
apiVersion: v1
kind: Pod
metadata:
  name: with-node-affinity
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: wolfcode.cn/framework-name
                operator: In
                values:
                  - spring
                  - springboot
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 1
          preference:
            matchExpressions:
              - key: another-node-label-key
                operator: In
                values:
                  - another-node-label-value
  containers:
    - name: with-node-affinity
      image: pauseyyf/pause
```

▪ podAffinity

podAffinity 基于 Pod 的标签来选择 Node，仅调度到满足条件Pod 所在的 Node 上，支持 podAffinity 和 podAntiAffinity。这个功能比较绕，以下面的例子为例：

- 如果一个 “Node 所在空间中包含至少一个带有 auth=oauth2 标签且运行中的 Pod”，那么可以调度到该 Node
- 不调度到 “包含至少一个带有 auth=jwt 标签且运行中 Pod”的 Node 上

```
apiVersion: v1
kind: Pod
metadata:
  name: with-pod-affinity
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: auth
                operator: In
                values:
                  - oauth2
            topologyKey: failure-domain.beta.kubernetes.io/zone
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchExpressions:
                - key: auth
                  operator: In
                  values:
                    - jwt
            topologyKey: kubernetes.io/hostname
  containers:
    - name: with-pod-affinity
      image: pauseyyf/pause
```

▪ 滚动更新

不建议使用 RollingUpdate，建议使用 OnDelete 模式，这样避免频繁更新 ds

▼ HPA 自动扩/缩容

通过观察 pod 的 cpu、内存使用率或自定义 metrics 指标进行自动的扩容或缩容 pod 的数量。

通常用于 Deployment，不适用于无法扩/缩容的对象，如 DaemonSet

控制管理器每隔30s（可以通过--horizontal-pod-autoscaler-sync-period修改）查询 metrics的资源使用情况

▪ 开启指标服务

下载 metrics-server 组件配置文件

```
wget https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml -O metrics-server-components.yaml
```

修改镜像地址为国内的地址

```
sed -i 's/k8s.gcr.io/metrics-server/registry.cn-hangzhou.aliyuncs.com/google_containers/g' metrics-server-components.yaml
```

修改容器的 tls 配置，不验证 tls，在 containers 的 args 参数中增加 --kubelet-insecure-tls 参数

安装组件

```
kubectctl apply -f metrics-server-components.yaml
```

查看 pod 状态

```
kubectctl get pods --all-namespaces | grep metrics
```

- **cpu、内存指标监控**

实现 cpu 或内存的监控，首先有个前提条件是该对象必须配置了 `resources.requests.cpu` 或 `resources.requests.memory` 才可以，可以配置当 `cpu/memory` 达到上述配置的百分比后进行扩容或缩容

创建一个 HPA：

1. 先准备一个好一个有做资源限制的 deployment
2. 执行命令 `kubectl autoscale deploy nginx-deploy --cpu-percent=20 --min=2 --max=5`
3. 通过 `kubectl get hpa` 可以获取 HPA 信息

测试：找到对应服务的 service，编写循环测试脚本提升内存与 cpu 负载
`while true; do wget -q -O- http://<ip:port> > /dev/null ; done`

可以通过多台机器执行上述命令，增加负载，当超过负载后可以查看 pods 的扩容情况 `kubectl get pods`

查看 pods 资源使用情况

`kubectl top pods`

扩容测试完成后，再关闭循环执行的指令，让 cpu 占用率降下来，然后过 5 分钟后查看自动缩容情况

- **自定义 metrics**

- 控制管理器开启 `-horizontal-pod-autoscaler-use-rest-clients`
- 控制管理器的 `-apiserver` 指向 [API Server Aggregator](#)
- 在 API Server Aggregator 中注册自定义的 metrics API

- ▼ **服务发布**

- ▼ **Service**

负责东西流量（同层级/内部服务网络通信）的通信

▼ Service 的定义

apiVersion: v1

kind: Service

metadata:

name: nginx-svc

labels:

app: nginx-svc

spec:

ports:

- name: http # service 端口配置的名称

protocol: TCP # 端口绑定的协议, 支持 TCP、UDP、SCTP, 默认为 TCP

port: 80 # service 自己的端口

targetPort: 9527 # 目标 pod 的端口

- name: https

port: 443

protocol: TCP

targetPort: 443

selector: # 选中当前 service 匹配哪些 pod, 对哪些 pod 的东西流量进行代理

app: nginx

▪ 命令操作

创建 service

kubectl create -f nginx-svc.yaml

查看 service 信息, 通过 service 的 cluster ip 进行访问

kubectl get svc

查看 pod 信息, 通过 pod 的 ip 进行访问

kubectl get po -o wide

创建其他 pod 通过 service name 进行访问 (推荐)

kubectl exec -it busybox -- sh

curl http://nginx-svc

默认在当前 namespace 中访问, 如果需要跨 namespace 访问 pod, 则在

service name 后面加上 .<namespace> 即可

curl http://nginx-svc.default

▪ Endpoint

▼ 代理 k8s 外部服务

实现方式：

1. 编写 service 配置文件时，不指定 selector 属性
2. 自己创建 endpoint

endpoint 配置：

apiVersion: v1

kind: Endpoints

metadata:

labels:

app: wolfcode-svc-external # 与 service 一致

name: wolfcode-svc-external # 与 service 一致

namespace: default # 与 service 一致

subsets:

- addresses:

- ip: <target ip> # 目标 ip 地址

ports: # 与 service 一致

- name: http

port: 80

protocol: TCP

- 各环境访问名称统一
- 访问 k8s 集群外的其他服务
- 项目迁移

▪ 反向代理外部域名

apiVersion: v1

kind: Service

metadata:

labels:

app: wolfcode-external-domain

name: wolfcode-external-domain

spec:

type: ExternalName

externalName: www.wolfcode.cn

▼ 常用类型

▪ ClusterIP

只能在集群内部使用，不配置类型的话默认就是 ClusterIP

▪ ExternalName

返回定义的 CNAME 别名，可以配置为域名

- NodePort

会在所有安装了 kube-proxy 的节点都绑定一个端口，此端口可以代理至对应的 Pod，集群外部可以使用任意节点 ip + NodePort 的端口号访问到集群中对应 Pod 中的服务。

当类型设置为 NodePort 后，可以在 ports 配置中增加 nodePort 配置指定端口，需要在下方的端口范围内，如果不指定会随机指定端口

端口范围：30000~32767

端口范围配置在 /usr/lib/systemd/system/kube-apiserver.service 文件中

- LoadBalancer

使用云服务商（阿里云、腾讯云等）提供的负载均衡器服务

- ▼ Ingress

Ingress 大家可以理解为也是一种 LB 的抽象，它的实现也是支持 nginx、haproxy 等负载均衡服务的

- ▼ 安装 ingress-nginx

<https://kubernetes.github.io/ingress-nginx/deploy/#using-helm>

- 添加 helm 仓库

- # 添加仓库

- helm repo add ingress-nginx <https://kubernetes.github.io/ingress-nginx>

- # 查看仓库列表

- helm repo list

- # 搜索 ingress-nginx

- helm search repo ingress-nginx

- 下载包

- # 下载安装包

- helm pull ingress-nginx/ingress-nginx

- 配置参数

- # 将下载好的安装包解压

- tar xf ingress-nginx-xxx.tgz

- # 解压后，进入解压完成的目录

- cd ingress-nginx

- # 修改 values.yaml

- 镜像地址：修改为国内镜像

- registry: registry.cn-hangzhou.aliyuncs.com

- image: google_containers/nginx-ingress-controller

- image: google_containers/kube-webhook-certgen

- tag: v1.3.0

- hostNetwork: true

- dnsPolicy: ClusterFirstWithHostNet

- 修改部署配置的 kind: DaemonSet

- nodeSelector:

- ingress: "true" # 增加选择器，如果 node 上有 ingress=true 就部署

- 将 admissionWebhooks.enabled 修改为 false

- 将 service 中的 type 由 LoadBalancer 修改为 ClusterIP，如果服务器是云平台才用 LoadBalancer

- 创建 Namespace

- # 为 ingress 专门创建一个 namespace

- kubectl create ns ingress-nginx

- 安装 ingress

- # 为需要部署 ingress 的节点上加标签

- kubectl label node k8s-node1 ingress=true

- # 安装 ingress-nginx

- helm install ingress-nginx ./ingress-nginx -n ingress-nginx

- ▼ 基本使用

- 创建一个 ingress

apiVersion: networking.k8s.io/v1

kind: Ingress # 资源类型为 Ingress

metadata:

name: wolrcode-inginx-ingress

annotations:

kubernetes.io/ingress.class: "nginx"

nginx.ingress.kubernetes.io/rewrite-target: /

spec:

rules: # ingress 规则配置, 可以配置多个

- host: k8s.wolrcode.cn # 域名配置, 可以使用通配符 *

http:

paths: # 相当于 nginx 的 location 配置, 可以配置多个

- pathType: Prefix # 路径类型, 按照路径类型进行匹配

ImplementationSpecific 需要指定 IngressClass, 具体匹配规则以 IngressClass 中的规则为准。Exact: 精确匹配, URL 需要与 path 完全匹配上, 且区分大小写的。Prefix: 以 / 作为分隔符来进行前缀匹配

backend:

service:

name: nginx-svc # 代理到哪个 service

port:

number: 80 # service 的端口

path: /api # 等价于 nginx 中的 location 的路径前缀匹配

▪ 多域名配置

apiVersion: networking.k8s.io/v1

kind: Ingress # 资源类型为 Ingress

metadata:

name: wolfcodes-ingress

annotations:

kubernetes.io/ingress.class: "nginx"

nginx.ingress.kubernetes.io/rewrite-target: /

spec:

rules: # ingress 规则配置，可以配置多个

- host: k8s.wolfcodes.cn # 域名配置，可以使用通配符 *

http:

paths: # 相当于 nginx 的 location 配置，可以配置多个

- pathType: Prefix # 路径类型，按照路径类型进行匹配

ImplementationSpecific 需要指定 IngressClass，具体匹配规则以 IngressClass

中的规则为准。Exact：精确匹配，URL需要与path完全匹配上，且区分大小写的。Prefix：以 / 作为分隔符来进行前缀匹配

backend:

service:

name: nginx-svc # 代理到哪个 service

port:

number: 80 # service 的端口

path: /api # 等价于 nginx 中的 location 的路径前缀匹配

- pathType: Exec # 路径类型，按照路径类型进行匹配

ImplementationSpecific 需要指定 IngressClass，具体匹配规则以 IngressClass

中的规则为准。Exact：精确匹配，URL需要与path完全匹配上，且区分大小写的。Prefix：以 / 作为分隔符来进行前缀匹配

backend:

service:

name: nginx-svc # 代理到哪个 service

port:

number: 80 # service 的端口

path: /

- host: api.wolfcodes.cn # 域名配置，可以使用通配符 *

http:

paths: # 相当于 nginx 的 location 配置，可以配置多个

- pathType: Prefix # 路径类型，按照路径类型进行匹配

ImplementationSpecific 需要指定 IngressClass，具体匹配规则以 IngressClass

中的规则为准。Exact：精确匹配，URL需要与path完全匹配上，且区分大小写的。Prefix：以 / 作为分隔符来进行前缀匹配

backend:

service:

name: nginx-svc # 代理到哪个 service

port:

```
number: 80 # service 的端口
path: /
```

▼ 配置与存储

▼ 配置管理

▼ ConfigMap

一般用于去存储 Pod 中应用所需的一些配置信息，或者环境变量，将配置于 Pod 分开，避免应为修改配置导致还需要重新构建 镜像与容器。

- 创建

使用 `kubectl create configmap -h` 查看示例，构建 configmap 对象

- 使用 ConfigMap

- 加密数据配置 Secret

与 ConfigMap 类似，用于存储配置信息，但是主要用于存储敏感信息、需要加密的信息，Secret 可以提供数据加密、解密功能。

在创建 Secret 时，要注意如果要加密的字符中，包含了有特殊字符，需要使用转义符转移，例如 \$ 转移后为 \ \$，也可以对特殊字符使用单引号描述，这样就不需要转移例如 `1$289*~!` 转换为 `'1$289*~!'`

- SubPath 的使用

使用 ConfigMap 或 Secret 挂载到目录的时候，会将容器中源目录给覆盖掉，此时我们可能只想覆盖目录中的某一个文件，但是这样的操作会覆盖整个文件，因此需要使用到 SubPath

配置方式：

1. 定义 volumes 时需要增加 items 属性，配置 key 和 path，且 path 的值不能从 / 开始
2. 在容器内的 volumeMounts 中增加 subPath 属性，该值与 volumes 中 items.path 的值相同

containers:

.....

volumeMounts:

- mountPath: /etc/nginx/nginx.conf # 挂载到哪里
- name: config-volume # 使用哪个 configmap 或 secret
- subPath: etc/nginx/nginx.conf # 与 volumes.[0].items.path 相同

volumes:

- configMap:
 - name: nginx-conf # configMap 名字
 - items: # subPath 配置
 - key: nginx.conf # configMap 中的文件名
 - path: etc/nginx/nginx.conf # subPath 路径

▼ 配置的热更新

我们通常会将项目的配置文件作为 configmap 然后挂载到 pod，那么如果更新 configmap 中的配置，会不会更新到 pod 中呢？

这得分成几种情况：

默认方式：会更新，更新周期是更新时间 + 缓存时间

subPath：不会更新

变量形式：如果 pod 中的一个变量是从 configmap 或 secret 中得到，同样也是不会更新的

对于 subPath 的方式，我们可以取消 subPath 的使用，将配置文件挂载到一个不存在的目录，避免目录的覆盖，然后再利用软连接的形式，将该文件链接到目标位置

但是如果目标位置原本就有文件，可能无法创建软链接，此时可以基于前面讲过的 postStart 操作执行删除命令，将默认的软链接删除即可

- 通过 edit 命令直接修改 configmap

- 通过 replace 替换

由于 configmap 我们创建通常都是基于文件创建，并不会编写 yaml 配置文件，因此修改时我们也是直接修改配置文件，而 replace 是没有 --from-file 参数的，因此无法实现基于源配置文件的替换，此时我们可以利用下方的命令实现

该命令的重点在于 --dry-run 参数，该参数的意思打印 yaml 文件，但不会将该文件发送给 apiserver，再结合 -oyaml 输出 yaml 文件就可以得到一个配置好但是没有发给 apiserver 的文件，然后再结合 replace 监听控制台输出得到 yaml 数据即可实现替换

```
kubectl create cm --from-file=nginx.conf --dry-run -oyaml | kubectl replace -f-
```

- 不可变的 Secret 和 ConfigMap

对于一些敏感服务的配置文件，在线上有时是不允许修改的，此时在配置 configmap 时可以设置 immutable: true 来禁止修改

▼ 持久化存储

▼ Volumes

▼ HostPath

将节点上的文件或目录挂载到 Pod 上，此时该目录会变成持久化存储目录，即使 Pod 被删除后重启，也可以重新加载到该目录，该目录下的文件不会丢失

▪ 配置文件

apiVersion: v1

kind: Pod

metadata:

name: test-pd

spec:

containers:

- image: nginx

name: nginx-volume

volumeMounts:

- mountPath: /test-pd # 挂载到容器的哪个目录

name: test-volume # 挂载哪个 volume

volumes:

- name: test-volume

hostPath:

path: /data # 节点中的目录

type: Directory # 检查类型，在挂载前对挂载目录做什么检查操作，有多种选项，默认为空字符串，不做任何检查

类型：

空字符串：默认类型，不做任何检查

DirectoryOrCreate：如果给定的 path 不存在，就创建一个 755 的空目录

Directory：这个目录必须存在

FileOrCreate：如果给定的文件不存在，则创建一个空文件，权限为 644

File：这个文件必须存在

Socket：UNIX 套接字，必须存在

CharDevice：字符设备，必须存在

BlockDevice：块设备，必须存在

▼ EmptyDir

EmptyDir 主要用于一个 Pod 中不同的 Container 共享数据使用的，由于只是在 Pod 内部使用，因此与其他 volume 比较大的区别是，当 Pod 如果被删除了，那么 emptyDir 也会被删除。

存储介质可以是任意类型，如 SSD、磁盘或网络存储。可以将 emptyDir.medium 设置为 Memory 让 k8s 使用 tmpfs（内存支持文件系统），速度比较快，但是重启 tmpfs 节点时，数据会被清除，且设置的大小会计入到 Container 的内存限制中。

- 配置文件

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pd
spec:
  containers:
  - image: nginx
    name: nginx-emptydir
    volumeMounts:
    - mountPath: /cache
      name: cache-volume
  volumes:
  - name: cache-volume
    emptyDir: {}
```

- ▼ NFS 挂载

nfs 卷能将 NFS (网络文件系统) 挂载到你的 Pod 中。不像 emptyDir 那样会在删除 Pod 的同时也会被删除，nfs 卷的内容在删除 Pod 时会被保存，卷只是被卸载。这意味着 nfs 卷可以被预先填充数据，并且这些数据可以在 Pod 之间共享。

- 安装 nfs

安装 nfs

```
yum install nfs-utils -y
```

启动 nfs

```
systemctl start nfs-server
```

查看 nfs 版本

```
cat /proc/fs/nfsd/versions
```

创建共享目录

```
mkdir -p /data/nfs
```

```
cd /data/nfs
```

```
mkdir rw
```

```
mkdir ro
```

设置共享目录 export

```
vim /etc/exports
```

```
/data/nfs/rw 192.168.113.0/24(rw,sync,no_subtree_check,no_root_squash)
```

```
/data/nfs/ro 192.168.113.0/24(ro,sync,no_subtree_check,no_root_squash)
```

重新加载

```
exportfs -f
```

```
systemctl reload nfs-server
```

到其他测试节点安装 nfs-utils 并加载测试

```
mkdir -p /mnt/nfs/rw
```

```
mount -t nfs 192.168.113.121:/data/nfs/rw /mnt/nfs/rw
```

- 配置文件

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pd
spec:
  containers:
  - image: nginx
    name: test-container
    volumeMounts:
    - mountPath: /my-nfs-data
      name: test-volume
  volumes:
  - name: test-volume
    nfs:
      server: my-nfs-server.example.com # 网络存储服务地址
      path: /my-nfs-volume # 网络存储路径
      readOnly: true # 是否只读
```

- ▼ PV 与 PVC

持久卷 (PersistentVolume, PV) 是集群中的一块存储，可以由管理员事先制备，或者使用[存储类 \(Storage Class\)](#) 来动态制备。持久卷是集群资源，就像节点也是集群资源一样。PV 持久卷和普通的 Volume 一样，也是使用卷插件来实现的，只是它们拥有独立于任何使用 PV 的 Pod 的生命周期。此 API 对象中记述了存储的实现细节，无论其背后是 NFS、iSCSI 还是特定于云平台的存储系统。

持久卷申领 (PersistentVolumeClaim, PVC) 表达的是用户对存储的请求。概念上与 Pod 类似。Pod 会耗用节点资源，而 PVC 申领会耗用 PV 资源。Pod 可以请求特定数量的资源（CPU 和内存）；同样 PVC 申领也可以请求特定的大小和访问模式（例如，可以要求 PV 卷能够以 ReadWriteOnce、ReadOnlyMany 或 ReadWriteMany 模式之一来挂载，参见[访问模式](#)）。

- ▼ 生命周期

- ▼ 构建

- 静态构建

- 集群管理员创建若干 PV 卷。这些卷对象带有真实存储的细节信息，并且对集群用户可用（可见）。PV 卷对象存在于 Kubernetes API 中，可供用户消费（使用）。

- 动态构建

- 如果集群中已经有的 PV 无法满足 PVC 的需求，那么集群会根据 PVC 自动构建一个 PV，该操作是通过 StorageClass 实现的。

- 想要实现这个操作，前提是 PVC 必须设置 StorageClass，否则会无法动态构建该 PV，可以通过启用 DefaultStorageClass 来实现 PV 的构建。

- **绑定**

当用户创建一个 PVC 对象后，主节点会监测新的 PVC 对象，并且寻找与之匹配的 PV 卷，找到 PV 卷后将二者绑定在一起。

如果找不到对应的 PV，则需要看 PVC 是否设置 StorageClass 来决定是否动态创建 PV，若没有配置，PVC 就会一直处于未绑定状态，直到有与之匹配的 PV 后才会申领绑定关系。

- **使用**

Pod 将 PVC 当作存储卷来使用，集群会通过 PVC 找到绑定的 PV，并为 Pod 挂载该卷。

Pod 一旦使用 PVC 绑定 PV 后，为了保护数据，避免数据丢失问题，PV 对象会受到保护，在系统中无法被删除。

- ▼ **回收策略**

当用户不再使用其存储卷时，他们可以从 API 中将 PVC 对象删除，从而允许该资源被回收再利用。PersistentVolume 对象的回收策略告诉集群，当其被从申领中释放时如何处理该数据卷。目前，数据卷可以被 Retained（保留）、Recycled（回收）或 Deleted（删除）。

- **保留（Retain）**

回收策略 Retain 使得用户可以手动回收资源。当 PersistentVolumeClaim 对象被删除时，PersistentVolume 卷仍然存在，对应的数据卷被视为“已释放（released）”。由于卷上仍然存在这前一个申领人的数据，该卷还不能用于其他申领。管理员可以通过下面的步骤来手动回收该卷：

1. 删除 PersistentVolume 对象。与之相关的、位于外部基础设施中的存储资产（例如 AWS EBS、GCE PD、Azure Disk 或 Cinder 卷）在 PV 删除之后仍然存在。
2. 根据情况，手动清除所关联的存储资产上的数据。
3. 手动删除所关联的存储资产。

如果你希望重用该存储资产，可以基于存储资产的定义创建新的 PersistentVolume 卷对象。

- **删除（Delete）**

对于支持 Delete 回收策略的卷插件，删除动作会将 PersistentVolume 对象从 Kubernetes 中移除，同时也会从外部基础设施（如 AWS EBS、GCE PD、Azure Disk 或 Cinder 卷）中移除所关联的存储资产。动态制备的卷会继承其 [StorageClass](#) 中设置的回收策略，该策略默认为 Delete。管理员需要根据用户的期望来配置 StorageClass；否则 PV 卷被创建之后必须要被编辑或者修补。

- 回收 (Recycle)

警告： 回收策略 Recycle 已被废弃。取而代之的建议方案是使用动态制备。

如果下层的卷插件支持，回收策略 Recycle 会在卷上执行一些基本的擦除（rm -rf /thevolume/*）操作，之后允许该卷用于新的 PVC 申领。

- ▼ PV

- ▼ 状态

- Available：空闲，未被绑定
 - Bound：已经被 PVC 绑定
 - Released：PVC 被删除，资源已回收，但是 PV 未被重新使用
 - Failed：自动回收失败

- 配置文件

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0001
spec:
  capacity:
    storage: 5Gi # pv 的容量
  volumeMode: Filesystem # 存储类型为文件系统
  accessModes: # 访问模式: ReadWriteOnce、ReadWriteMany、ReadOnlyMany
    - ReadWriteOnce # 可被单节点独写
  persistentVolumeReclaimPolicy: Recycle # 回收策略
  storageClassName: slow # 创建 PV 的存储类名，需要与 pvc 的相同
  mountOptions: # 加载配置
    - hard
    - nfsvers=4.1
  nfs: # 连接到 nfs
    path: /data/nfs/rw/test-pv # 存储路径
    server: 192.168.113.121 # nfs 服务地址
```

- ▼ PVC

- Pod 绑定 PVC

在 pod 的挂载容器配置中，增加 pvc 挂载
containers:

```
.....  
volumeMounts:  
  - mountPath: /tmp/pvc  
    name: nfs-pvc-test  
volumes:  
  - name: nfs-pvc-test  
    persistentVolumeClaim:  
      claimName: nfs-pvc # pvc 的名称
```

- 配置文件

```
apiVersion: v1  
kind: PersistentVolumeClaim  
metadata:  
  name: nfs-pvc  
spec:  
  accessModes:  
    - ReadWriteOnce # 权限需要与对应的 pv 相同  
  volumeMode: Filesystem  
  resources:  
    requests:  
      storage: 8Gi # 资源可以小于 pv 的，但是不能大于，如果大于就会匹配不到 pv  
  storageClassName: slow # 名字需要与对应的 pv 相同  
# selector: # 使用选择器选择对应的 pv  
#   matchLabels:  
#     release: "stable"  
#   matchExpressions:  
#     - {key: environment, operator: In, values: [dev]}
```

- ▼ StorageClass

k8s 中提供了一套自动创建 PV 的机制，就是基于 StorageClass 进行的，通过 StorageClass 可以实现仅仅配置 PVC，然后交由 StorageClass 根据 PVC 的需求动态创建 PV。

- 制备器 (Provisioner)

 StorageClass 制备器.md

每个 StorageClass 都有一个制备器 (Provisioner)，用来决定使用哪个卷插件制备 PV。

- ▼ NFS 动态制备案例

- nfs-provisioner

apiVersion: apps/v1

kind: Deployment

metadata:

name: nfs-client-provisioner

namespace: kube-system

labels:

app: nfs-client-provisioner

spec:

replicas: 1

strategy:

type: Recreate

selector:

matchLabels:

app: nfs-client-provisioner

template:

metadata:

labels:

app: nfs-client-provisioner

spec:

serviceAccountName: nfs-client-provisioner

containers:

- name: nfs-client-provisioner

image: quay.io/external_storage/nfs-client-provisioner:latest

volumeMounts:

- name: nfs-client-root

mountPath: /persistentvolumes

env:

- name: PROVISIONER_NAME

value: fuseim.pri/ifs

- name: NFS_SERVER

value: 192.168.113.121

- name: NFS_PATH

value: /data/nfs/rw

volumes:

- name: nfs-client-root

nfs:

server: 192.168.113.121

path: /data/nfs/rw

- StorageClass 配置

apiVersion: storage.k8s.io/v1

kind: StorageClass

metadata:

name: managed-nfs-storage

namespace: kube-system

provisioner: fuseim.pri/ifs # 外部制备器提供者, 编写为提供者的名称

parameters:

archiveOnDelete: "false" # 是否存档, false 表示不存档, 会删除
oldPath 下面的数据, true 表示存档, 会重命名路径

reclaimPolicy: Retain # 回收策略, 默认为 Delete 可以配置为 Retain

volumeBindingMode: Immediate # 默认为 Immediate, 表示创建 PVC 立
即进行绑定, 只有 azuredisk 和 AWSelastiblockstore 支持其他值

▪ RBAC 配置

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: nfs-client-provisioner
  namespace: kube-system
---
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: nfs-client-provisioner-runner
  namespace: kube-system
rules:
  - apiGroups: [""]
    resources: ["persistentvolumes"]
    verbs: ["get", "list", "watch", "create", "delete"]
  - apiGroups: [""]
    resources: ["persistentvolumeclaims"]
    verbs: ["get", "list", "watch", "update"]
  - apiGroups: ["storage.k8s.io"]
    resources: ["storageclasses"]
    verbs: ["get", "list", "watch"]
  - apiGroups: [""]
    resources: ["events"]
    verbs: ["create", "update", "patch"]
---
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: run-nfs-client-provisioner
  namespace: kube-system
subjects:
  - kind: ServiceAccount
    name: nfs-client-provisioner
    namespace: default
roleRef:
  kind: ClusterRole
  name: nfs-client-provisioner-runner
  apiGroup: rbac.authorization.k8s.io
---
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: leader-locking-nfs-client-provisioner
  namespace: kube-system
```

```

rules:
  - apiGroups: [""]
    resources: ["endpoints"]
    verbs: ["get", "list", "watch", "create", "update", "patch"]
---
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: leader-locking-nfs-client-provisioner
  namespace: kube-system
subjects:
  - kind: ServiceAccount
    name: nfs-client-provisioner
roleRef:
  kind: Role
  name: leader-locking-nfs-client-provisioner
  apiGroup: rbac.authorization.k8s.io

```

▼ PVC 处于 Pending 状态

在 k8s 1.20 之后，出于对性能和统一 apiserver 调用方式的初衷，移除了对 SelfLink 的支持，而默认上面指定的 provisioner 版本需要 SelfLink 功能，因此 PVC 无法进行自动制备。

▪ 配置 SelfLink

修改 apiserver 配置文件

```
vim /etc/kubernetes/manifests/kube-apiserver.yaml
```

```

spec:
  containers:
    - command:
      - kube-apiserver
      - --feature-gates=RemoveSelfLink=false # 新增该行
      .....

```

修改后重新应用该配置

```
kubectl apply -f /etc/kubernetes/manifests/kube-apiserver.yaml
```

▪ 不需要 SelfLink 的 provisioner

将 provisioner 修改为如下镜像之一即可

gcr.io/k8s-staging-sig-storage/nfs-subdir-external-provisioner:v4.0.0

registry.cn-beijing.aliyuncs.com/pylixm/nfs-subdir-external-provisioner:v4.0.0

- PVC 测试配置

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: auto-pv-test-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 300Mi
  storageClassName: managed-nfs-storage
```

- ▼ 高级调度

- ▼ CronJob 计划任务

在 k8s 中周期性运行计划任务，与 linux 中的 crontab 相同

注意点：CronJob 执行的时间是 controller-manager 的时间，所以一定要确保 controller-manager 时间是准确的，另外 cronjob

- cron 表达式

```
# |_____ 分钟 (0 - 59)
# | |_____ 小时 (0 - 23)
# | | |_____ 月的某天 (1 - 31)
# | | | |_____ 月份 (1 - 12)
# | | | | |_____ 周的某天 (0 - 6) (周日到周一；在某些系统上，7 也是星期日)
# | | | | |_____ 或者是 sun, mon, tue, web, thu, fri, sat
# | | | | |
# * * * * *
```

■ 配置文件

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello
spec:
  concurrencyPolicy: Allow # 并发调度策略: Allow 允许并发调度, Forbid: 不允许并发执行, Replace: 如
  failedJobsHistoryLimit: 1 # 保留多少个失败的任务
  successfulJobHistoryLimit: 3 # 保留多少个成功的任务
  suspend: false # 是否挂起任务, 若为 true 则该任务不会执行
  # startingDeadlineSeconds: 30 # 间隔多长时间检测失败的任务并重新执行, 时间不能小于 10
  schedule: "* * * * *" # 调度策略
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: hello
              image: busybox:1.28
              imagePullPolicy: IfNotPresent
              command:
                - /bin/sh
                - -c
                - date; echo Hello from the Kubernetes cluster
          restartPolicy: OnFailure
```

■ 初始化容器 InitContainer

在真正的容器启动之前, 先启动 InitContainer, 在初始化容器中完成真实容器所需的初始化操作, 完成后再启动真实的容器。

相对于 postStart 来说, 首先 InitController 能够保证一定在 EntryPoint 之前执行, 而 postStart 不能, 其次 postStart 更适合去执行一些命令操作, 而 InitController 实际就是一个容器, 可以在其他基础容器环境下执行更复杂的初始化功能。

在 pod 创建的模板中配置 initContainers 参数:

```
spec:
  initContainers:
    - image: nginx
      imagePullPolicy: IfNotPresent
      command: ["sh", "-c", "echo 'inited;' >> ~/.init"]
      name: init-test
```

▼ 污点和容忍

k8s 集群中可能管理着非常庞大的服务器, 这些服务器可能是各种各样不同类型的, 比如机房、地理位置、配置等, 有些是计算型节点, 有些是存储型节点, 此时我们希望能更好的将 pod 调度到与之需求更匹配的节点上。

此时就需要用到污点 (Taint) 和容忍 (Toleration), 这些配置都是 key: value 类型的。

▼ 污点 (Taint)

污点：是标注在节点上的，当我们在一个节点上打上污点以后，k8s 会认为尽量不要将 pod 调度到该节点上，除非该 pod 上面表示可以容忍该污点，且一个节点可以打多个污点，此时则需要 pod 容忍所有污点才会被调度该节点。

为节点打上污点

```
kubectl taint node k8s-master key=value:NoSchedule
```

移除污点

```
kubectl taint node k8s-master key=value:NoSchedule-
```

查看污点

```
kubectl describe no k8s-master
```

污点的影响：

NoSchedule：不能容忍的 pod 不能被调度到该节点，但是已经存在的节点不会被驱逐

NoExecute：不能容忍的节点会被立即清除，能容忍且没有配置 **tolerationSeconds** 属性，则可以一直运行，设置了 **tolerationSeconds: 3600** 属性，则该 pod 还能继续在该节点运行 3600 秒

- NoSchedule

- 如果不能容忍该污点，那么 Pod 就无法调度到该节点上

- NoExecute

- 如果 Pod 不能忍受这类污点，Pod 会马上被驱逐。
 - 如果 Pod 能够忍受这类污点，但是在容忍度定义中没有指定 **tolerationSeconds**，则 Pod 还会一直在这个节点上运行。
 - 如果 Pod 能够忍受这类污点，而且指定了 **tolerationSeconds**，则 Pod 还能在这个节点上继续运行这个指定的时间长度。

▼ 容忍 (Toleration)

容忍：是标注在 pod 上的，当 pod 被调度时，如果没有配置容忍，则该 pod 不会被调度到有污点的节点上，只有该 pod 上标注了满足某个节点的所有污点，则会被调度到这些节点

pod 的 spec 下面配置容忍

tolerations:

- key: "污点的 key"

- value: "污点的 value"

- effect: "NoSchedule" # 污点产生的影响

- operator: "Equal" # 要是 value 与污点的 value 要相等，也可以设置为 Exists 表示存在 key 即可，此时可以不用配置 value

- Equal

比较操作类型为 Equal，则意味着必须与污点值做匹配，key/value都必须相同，才表示能够容忍该污点

- Exists

容忍与污点的比较只比较 key，不比较 value，不关心 value 是什么东西，只要 key 存在，就表示可以容忍。

- ▼ 亲和力 (Affinity)

- ▼ NodeAffinity

节点亲和力：进行 pod 调度时，优先调度到符合条件的亲和力节点上

- RequiredDuringSchedulingIgnoredDuringExecution

硬亲和力，即支持必须部署在指定的节点上，也支持必须不部署在指定的节点上

- PreferredDuringSchedulingIgnoredDuringExecution

软亲和力：尽量部署在满足条件的节点上，或尽量不要部署在被匹配的节点上

- ▼ 应用

- ▼ 匹配类型

- In

部署在满足条件的节点上

- NotIn

匹配不在条件中的节点，实现节点反亲和性

- Exists

只要存在 key 名字就可以，不关心值是什么

- DoesNotExist

匹配指定 key 名不存在的节点，实现节点反亲和性

- Gt

value 为数值，且节点上的值小于指定的条件

- Lt

value 为数值，且节点上的值大于指定条件

■ 配置模板

```
apiVersion: v1
kind: Pod
metadata:
  name: with-node-affinity
spec:
  affinity: # 亲和力配置
    nodeAffinity: # 节点亲和力
      requiredDuringSchedulingIgnoredDuringExecution: # 节点必须匹配下方配置
        nodeSelectorTerms: # 选择器
          - matchExpressions: # 匹配表达式
              - key: topology.kubernetes.io/zone # 匹配 label 的 key
                operator: In # 匹配方式，只要匹配成功下方的一个 value 即可
                values:
                  - antarctica-east1 # 匹配的 value
                  - antarctica-west1 # 匹配的 value
            preferredDuringSchedulingIgnoredDuringExecution: # 节点尽量匹配下方配置
              - weight: 1 # 权重[1,100]，按照匹配规则对所有节点累加权重，最终之和会加入优先级评分，
                preference:
                  matchExpressions: # 匹配表达式
                    - key: another-node-label-key # label 的 key
                      operator: In # 匹配方式，满足一个即可
                      values:
                        - another-node-label-value # 匹配的 value
              # - weight: 20
              # .....
        containers:
          - name: with-node-affinity
            image: pause:2.0
```

▼ PodAffinity

Pod 亲和力：将与指定 pod 亲和力相匹配的 pod 部署在同一节点。

- RequiredDuringSchedulingIgnoredDuringExecution
必须将应用部署在一块
- PreferredDuringSchedulingIgnoredDuringExecution
尽量将应用部署在一块

■ 配置模板

```
apiVersion: v1
kind: Pod
metadata:
  name: with-pod-affinity
spec:
  affinity: # 亲和力配置
  podAffinity: # pod 亲和力配置
    requiredDuringSchedulingIgnoredDuringExecution: # 当前 pod 必须匹配到对应条件 pod 所在
      - labelSelector: # 标签选择器
        matchExpressions: # 匹配表达式
          - key: security # 匹配的 key
            operator: In # 匹配方式
            values: # 匹配其中的一个 value
          - S1
        topologyKey: topology.kubernetes.io/zone
    podAntiAffinity: # pod 反亲和力配置
      preferredDuringSchedulingIgnoredDuringExecution: # 尽量不要将当前节点部署到匹配下列参数
        - weight: 100 # 权重
          podAffinityTerm: # pod 亲和力配置条件
            labelSelector: # 标签选择器
              matchExpressions: # 匹配表达式
                - key: security # 匹配的 key
                  operator: In # 匹配的方式
                  values:
                    - S2 # 匹配的 value
            topologyKey: topology.kubernetes.io/zone
  containers:
    - name: with-pod-affinity
      image: pause:2.0
```

<

>

▼ PodAntiAffinity

Pod 反亲和力：根据策略尽量部署或不部署到一块

■ RequiredDuringSchedulingIgnoredDuringExecution

不要将应用与之匹配的部署到一块

```
podAffinity:
  requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchExpressions:
          - key: security
            operator: In
            values:
              - S1
        topologyKey: topology.kubernetes.io/zone
```

■ PreferredDuringSchedulingIgnoredDuringExecution

尽量不要将应用部署到一块

▼ 身份认证与权限

Kubernetes 中提供了良好的多租户认证管理机制，如 RBAC、ServiceAccount 还有各种策略等。

通过该文件可以看到已经配置了 RBAC 访问控制
`/usr/lib/systemd/system/kube-apiserver.service`

▼ 认证

所有 Kubernetes 集群有两类用户：由 Kubernetes 管理的 **Service Accounts**（服务账户）和（Users Accounts）普通账户。

普通账户是假定被外部或独立服务管理的，由管理员分配 keys，用户像使用 Keystone 或 google 账号一样，被存储在包含 usernames 和 passwords 的 list 的文件里。

需要注意：在 Kubernetes 中不能通过 API 调用将普通用户添加到集群中。

- 普通帐户是针对（人）用户的，服务账户针对 Pod 进程。
- 普通帐户是全局性。在集群所有 namespaces 中，名称具有惟一性。
- 通常，群集的普通帐户可以与企业数据库同步，新的普通帐户创建需要特殊权限。服务账户创建目的是更轻量化，允许集群用户为特定任务创建服务账户。
- 普通帐户和服务账户的审核注意事项不同。
- 对于复杂系统的配置包，可以包括对该系统的各种组件的服务账户的定义。

▪ User Accounts

▼ Service Accounts

▼ Service Account 自动化

- **Service Account Admission Controller**

通过 [Admission Controller](#) 插件来实现对 pod 修改，它是 apiserver 的一部分。创建或更新 pod 时会同步进行修改 pod。当插件处于激活状态（在大多数发行版中都默认情况）创建或修改 pod 时，会按以下操作执行：

1. 如果 pod 没有设置 ServiceAccount，则将 ServiceAccount 设置为 default。
2. 确保 pod 引用的 ServiceAccount 存在，否则将会拒绝请求。
3. 如果 pod 不包含任何 ImagePullSecrets，则将 ServiceAccount 的 ImagePullSecrets 会添加到 pod 中。
4. 为包含 API 访问的 Token 的 pod 添加了一个 volume。
5. 把 volumeSource 添加到安装在 pod 的每个容器中，挂载在 /var/run/secrets/kubernetes.io/serviceaccount。

- **Token Controller**

TokenController 作为 controller-manager 的一部分运行。异步行为：

- 观察 serviceAccount 的创建，并创建一个相应的 Secret 来允许 API 访问。
- 观察 serviceAccount 的删除，并删除所有相应的 ServiceAccountToken Secret
- 观察 secret 添加，并确保关联的 ServiceAccount 存在，并在需要时向 secret 中添加一个 Token。
- 观察 secret 删除，并在需要时对应 ServiceAccount 的关联

- **Service Account Controller**

Service Account Controller 在 namespaces 里管理服务 Account，并确保每个有效的 namespaces 中都存在一个名为 "default" 的 ServiceAccount。

- ▼ **授权 (RBAC)**

- ▼ **Role**

代表一个角色，会包含一组权限，没有拒绝规则，只是附加允许。它是 Namespace 级别的资源，只能作用与 Namespace 之内。

查看已有的角色信息

kubectl get role -n ingress-nginx -oyaml

- 配置文件

apiVersion: [rbac.authorization.k8s.io/v1](https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.18/#role-v1-rbac.authorization.k8s.io)

kind: Role

metadata:

labels:

[app.kubernetes.io/name](https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.18/#ingress-nginx-label): ingress-nginx

[app.kubernetes.io/part-of](https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.18/#ingress-nginx-label): ingress-nginx

name: nginx-ingress

namespace: ingress-nginx

roles:

- apiGroups:

- ""

- resources:

- configmaps

- pods

- secrets

- namespaces

- verbs:

- get

- apiGroups:

- ""

- resourceNames:

- ingress-controller-label-nginx

- resources:

- configmaps

- verbs:

- get

- update

- apiGroups:

- ""

- resources:

- configmaps

- verbs:

- create

- ClusterRole

功能与 Role 一样，区别是资源类型为集群类型，而 Role 只在 Namespace

查看某个集群角色的信息

kubectl get clusterrole view -oyaml

▼ RoleBinding

Role 或 ClusterRole 只是用于制定权限集合，具体作用与什么对象上，需要使用 RoleBinding 来进行绑定。

作用于 Namespace 内，可以将 Role 或 ClusterRole 绑定到 User、Group、Service Account 上。

查看 rolebinding 信息

```
kubectl get rolebinding --all-namespaces
```

查看指定 rolebinding 的配置信息

```
kubectl get rolebinding <role_binding_name> --all-namespaces -oyaml
```

▪ 配置文件

```
apiVersion: rbac.authorization.k8s.io/v1
```

```
kind: RoleBinding
```

```
metadata:
```

```
.....
```

```
roleRef:
```

```
  apiGroup: rbac.authorization.k8s.io
```

```
  kind: Role
```

```
  name nginx-ingress-role
```

```
subjects:
```

```
- kind: ServiceAccount
```

```
  name: nginx-ingress-serviceaccount
```

```
  namespace: ingress-nginx
```

▪ ClusterRoleBinding

与 RoleBinding 相同，但是作用于集群之上，可以绑定到该集群下的任意 User、Group 或 Service Account

▼ 4 IV. 运维管理篇

▼ Helm 包管理器

- 什么是 Helm?

Kubernetes 包管理器

Helm 是查找、分享和使用软件构件 Kubernetes 的最优方式。

Helm 管理名为 chart 的 Kubernetes 包的工具。Helm 可以做以下的事情：

- 从头开始创建新的 chart
- 将 chart 打包成归档(tgz)文件
- 与存储 chart 的仓库进行交互
- 在现有的 Kubernetes 集群中安装和卸载 chart
- 管理与 Helm 一起安装的 chart 的发布周期

对于Helm，有三个重要的概念：

1. *chart* 创建Kubernetes应用程序所必需的一组信息。
2. *config* 包含了可以合并到打包的chart中的配置信息，用于创建一个可发布的对象。
3. *release* 是一个与特定配置相结合的chart的运行实例。

- ▼ Helm 架构

- ▼ 重要概念

- chart

- chart* 创建 Kubernetes 应用程序所必需的一组信息。

- config

- config* 包含了可以合并到打包的 chart 中的配置信息，用于创建一个可发布的对象。

- release

- release* 是一个与特定配置相结合的 chart 的运行实例。

- ▼ 组件

- Helm 客户端

Helm 客户端 是终端用户的命令行客户端。负责以下内容：

- 本地 chart 开发
- 管理仓库
- 管理发布
- 与 Helm 库建立接口
 - 发送安装的 chart
 - 发送升级或卸载现有发布请求

- Helm 库

Helm 库 提供执行所有 Helm 操作的逻辑。与 Kubernetes API 服务交互并提供以下功能：

- 结合 chart 和配置来构建版本
- 将 chart 安装到 Kubernetes 中，并提供后续发布对象
- 与 Kubernetes 交互升级和卸载 chart

独立的 Helm 库封装了 Helm 逻辑以便不同的客户端可以使用它。

- ▼ 安装 Helm

<https://helm.sh/docs/intro/install>

- 下载二进制文件

<https://get.helm.sh/helm-v3.2.3-linux-amd64.tar.gz>

- 解压(tar -zxvf helm-v3.10.2-linux-amd64.tar.gz)

- 将解压目录下的 helm 程序移动到 usr/local/bin/helm

- 添加阿里云 helm 仓库

- ▼ Helm 的常用命令

- ▼ helm repo

- 列出、增加、更新、删除 chart 仓库

- ▼ helm search

- 使用关键词搜索 chart

- ▼ helm pull
 - 拉取远程仓库中的 chart 到本地
- ▼ helm create
 - 在本地创建新的 chart
- ▼ helm dependency
 - 管理 chart 依赖
- ▼ helm install
 - 安装 chart
- ▼ helm list
 - 列出所有 release
- ▼ helm lint
 - 检查 chart 配置是否有误
- ▼ helm package
 - 打包本地 chart
- ▼ helm rollback
 - 回滚 release 到历史版本
- ▼ helm uninstall
 - 卸载 release
- ▼ helm upgrade
 - 升级 release
- ▼ chart 详解
 - 目录结构

```
mychart
├── Chart.yaml
├── charts # 该目录保存其他依赖的 chart (子 chart)
├── templates # chart 配置模板, 用于渲染最终的 Kubernetes YAML 文件
│   ├── NOTES.txt # 用户运行 helm install 时候的提示信息
│   ├── _helpers.tpl # 用于创建模板时的帮助类
│   ├── deployment.yaml # Kubernetes deployment 配置
│   ├── ingress.yaml # Kubernetes ingress 配置
│   ├── service.yaml # Kubernetes service 配置
│   ├── serviceaccount.yaml # Kubernetes serviceaccount 配置
│   └── tests
│       └── test-connection.yaml
└── values.yaml # 定义 chart 模板中的自定义配置的默认值, 可以在执行 helm install 或 helm update 的时候覆
```

▼ Redis chart 实践

▪ 修改 helm 源

查看默认仓库

```
helm repo list
```

添加仓库

```
helm repo add bitnami https://charts.bitnami.com/bitnami
```

```
helm repo add aliyun https://apphub.aliyuncs.com/stable
```

```
helm repo add azure http://mirror.azure.cn/kubernetes/charts
```

▪ 搜索 redis chart

搜索 redis chart

```
helm search repo redis
```

查看安装说明

```
helm show readme bitnami/redis
```

▪ 修改配置安装

先将 chart 拉到本地

```
helm pull bitnami/redis
```

解压后，修改 values.yaml 中的参数

```
tar -xvf redis-17.4.3.tgz
```

修改 storageClass 为 managed-nfs-storage

设置 redis 密码 password

修改集群架构 architecture，默认是主从（replication，3个节点），可以修改为 standalone 单机模式

修改实例存储大小 persistence.size 为需要的大小

修改 service.nodePorts.redis 向外暴露端口，范围 <30000-32767>

安装操作

创建命名空间

```
kubectl create namespace redis
```

安装

```
cd ../
```

```
helm install redis ./redis -n redis
```

▪ 查看安装情况

查看 helm 安装列表

```
helm list
```

查看 redis 命名空间下所有对象信息

```
kubectl get all -n redis
```

- 升级与回滚

要想升级 chart 可以修改本地的 chart 配置并执行：

```
helm upgrade [RELEASE] [CHART] [flags]
helm upgrade redis ./redis
```

使用 helm ls 的命令查看当前运行的 chart 的 release 版本，并使用下面的命令回滚到历史版本：

```
helm rollback <RELEASE> [REVISION] [flags]
```

查看历史

```
helm history redis
```

回退到上一版本

```
helm rollback redis
```

回退到指定版本

```
helm rollback redis 3
```

- helm 卸载 redis

```
helm delete redis -n redis
```

- ▼ k8s 集群监控

- ▼ 监控方案

- Heapster

Heapster 是容器集群监控和性能分析工具，天然的支持Kubernetes 和 CoreOS。

Kubernetes 有个出名的监控 agent---cAdvisor。在每个kubernetes Node 上都会运行 cAdvisor，它会收集本机以及容器的监控数据 (cpu,memory,filesystem,network,uptime)。

在较新的版本中，K8S 已经将 cAdvisor 功能集成到 kubelet 组件中。每个 Node 节点可以直接进行 web 访问。

- Weave Scope

[Weave Scope](#) 可以监控 kubernetes 集群中的一系列资源的状态、资源使用情况、应用拓扑、scale、还可以直接通过浏览器进入容器内部调试等，其提供的功能包括：

- 交互式拓扑界面
- 图形模式和表格模式
- 过滤功能
- 搜索功能
- 实时度量
- 容器排错
- 插件扩展

- Prometheus

Prometheus 是一套开源的监控系统、报警、时间序列的集合，最初由 SoundCloud 开发，后来随着越来越多公司的使用，于是便独立成开源项目。自此以后，许多公司和组织都采用了 Prometheus 作为监控告警工具。

- ▼ Prometheus 监控 k8s

- ▼ 自定义配置

- 创建 ConfigMap 配置

```
# 创建 prometheus-config.yml
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-config
data:
  prometheus.yml: |
    global:
      scrape_interval: 15s
      evaluation_interval: 15s
    scrape_configs:
      - job_name: 'prometheus'
        static_configs:
          - targets: ['localhost:9090']

# 创建 configmap
kubectl create -f prometheus-config.yml
```

- 部署 Prometheus

```
# 创建 prometheus-deploy.yml
apiVersion: v1
kind: Service
metadata:
  name: prometheus
  labels:
    name: prometheus
spec:
  ports:
    - name: prometheus
      protocol: TCP
      port: 9090
      targetPort: 9090
  selector:
    app: prometheus
  type: NodePort
---
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    name: prometheus
  name: prometheus
spec:
  replicas: 1
  selector:
    matchLabels:
      app: prometheus
  template:
    metadata:
      labels:
        app: prometheus
    spec:
      containers:
        - name: prometheus
          image: prom/prometheus:v2.2.1
          command:
            - "/bin/prometheus"
          args:
            - "--config.file=/etc/prometheus/prometheus.yml"
          ports:
            - containerPort: 9090
              protocol: TCP
          volumeMounts:
```

```
- mountPath: "/etc/prometheus"
  name: prometheus-config
volumes:
- name: prometheus-config
  configMap:
    name: prometheus-config
```

创建部署对象

```
kubectl create -f prometheus-deploy.yml
```

查看是否在运行中

```
kubectl get pods -l app=prometheus
```

获取服务信息

```
kubectl get svc -l name=prometheus
```

通过 http://节点ip:端口 进行访问

▪ 配置访问权限

```
# 创建 prometheus-rbac-setup.yml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: prometheus
rules:
- apiGroups: ["" ]
  resources:
    - nodes
    - nodes/proxy
    - services
    - endpoints
    - pods
  verbs: ["get", "list", "watch"]
- apiGroups:
  - extensions
  resources:
  - ingresses
  verbs: ["get", "list", "watch"]
- nonResourceURLs: ["/metrics"]
  verbs: ["get"]
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: prometheus
  namespace: default
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: prometheus
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: prometheus
subjects:
- kind: ServiceAccount
  name: prometheus
  namespace: default

# 创建资源对象
kubectl create -f prometheus-rbac-setup.yml
```

修改 prometheus-deploy.yml 配置文件

spec:

replicas: 1

template:

metadata:

labels:

app: prometheus

spec:

serviceAccountName: prometheus

serviceAccount: prometheus

升级 prometheus-deployment

kubectl apply -f prometheus-deployment.yml

查看 pod

kubectl get pods -l app=prometheus

查看 serviceaccount 认证证书

kubectl exec -it <pod name> -- ls

/var/run/secrets/kubernetes.io/serviceaccount/

- 服务发现配置

配置 job, 帮助 prometheus 找到所有节点信息, 修改 prometheus-config.yml 增加为如下内容

data:

prometheus.yml: |

global:

scrape_interval: 15s

evaluation_interval: 15s

scrape_configs:

- job_name: 'prometheus'

static_configs:

- targets: ['localhost:9090']

- job_name: 'kubernetes-nodes'

tls_config:

ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt

bearer_token_file: /var/run/secrets/kubernetes.io/serviceaccount/token

kubernetes_sd_configs:

- role: node

- job_name: 'kubernetes-service'

tls_config:

ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt

bearer_token_file: /var/run/secrets/kubernetes.io/serviceaccount/token

kubernetes_sd_configs:

- role: service

- job_name: 'kubernetes-endpoints'

tls_config:

ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt

bearer_token_file: /var/run/secrets/kubernetes.io/serviceaccount/token

kubernetes_sd_configs:

- role: endpoints

- job_name: 'kubernetes-ingress'

tls_config:

ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt

bearer_token_file: /var/run/secrets/kubernetes.io/serviceaccount/token

kubernetes_sd_configs:

- role: ingress

- job_name: 'kubernetes-pods'

tls_config:

ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt

bearer_token_file: /var/run/secrets/kubernetes.io/serviceaccount/token

kubernetes_sd_configs:

- role: pod
升级配置
kubectl apply -f prometheus-config.yml

获取 prometheus pod
kubectl get pods -l app=prometheus

删除 pod
kubectl delete pods <pod name>

查看 pod 状态
kubectl get pods

重新访问 ui 界面

- 系统时间同步

查看系统时间
date

同步网络时间
ntpdate cn.pool.ntp.org

▼ 监控 k8s 集群

往 prometheus-config.yml 中追加如下配置

```
- job_name: 'kubernetes-kubelet'
  scheme: https
  tls_config:
    ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
    bearer_token_file: /var/run/secrets/kubernetes.io/serviceaccount/token
  kubernetes_sd_configs:
    - role: node
  relabel_configs:
    - action: labelmap
      regex: __meta_kubernetes_node_label_(.+)
    - target_label: __address__
      replacement: kubernetes.default.svc:443
    - source_labels: [__meta_kubernetes_node_name]
      regex: (.+)
      target_label: __metrics_path__
      replacement: /api/v1/nodes/${1}/proxy/metrics
```

升级资源

kubectl apply -f prometheus-config.yml

重新构建应用

kubectl delete pods <pod name>

利用指标获取当前节点中 pod 的启动时间

kubelet_pod_start_latency_microseconds{quantile="0.99"}

计算平均时间

kubelet_pod_start_latency_microseconds_sum /
kubelet_pod_start_latency_microseconds_count

- 从 kubelet 获取节点容器资源使用情况

修改配置文件，增加如下内容，并更新服务

- job_name: 'kubernetes-cadvisor'

scheme: https

tls_config:

- ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt

bearer_token_file:

/var/run/secrets/kubernetes.io/serviceaccount/token

kubernetes_sd_configs:

- role: node

relabel_configs:

- target_label: __address__

replacement: kubernetes.default.svc:443

- source_labels: [__meta_kubernetes_node_name]

regex: (.+)

target_label: __metrics_path__

replacement: /api/v1/nodes/\${1}/proxy/metrics/cadvisor

- action: labelmap

regex: __meta_kubernetes_node_label_(.+)

- **Exporter 监控资源使用情况**

创建 node-exporter-daemonset.yml 文件

apiVersion: apps/v1

kind: DaemonSet

metadata:

name: node-exporter

spec:

template:

metadata:

annotations:

prometheus.io/scrape: 'true'

prometheus.io/port: '9100'

prometheus.io/path: 'metrics'

labels:

app: node-exporter

name: node-exporter

spec:

containers:

- image: prom/node-exporter

imagePullPolicy: IfNotPresent

name: node-exporter

ports:

- containerPort: 9100

hostPort: 9100

name: scrape

hostNetwork: true

hostPID: true

创建 daemonset

kubectl create -f node-exporter-daemonset.yml

查看 daemonset 运行状态

kubectl get daemonsets -l app=node-exporter

查看 pod 状态

kubectl get pods -l app=node-exporter

修改配置文件，增加监控采集任务

- job_name: 'kubernetes-pods'

kubernetes_sd_configs:

- role: pod

relabel_configs:

- source_labels:

[__meta_kubernetes_pod_annotation_prometheus_io_scrape]

action: keep

```

    regex: true
  - source_labels:
    [__meta_kubernetes_pod_annotation_prometheus_io_path]
    action: replace
    target_label: __metrics_path__
    regex: (.+)
  - source_labels: [__address__,
__meta_kubernetes_pod_annotation_prometheus_io_port]
    action: replace
    regex: ([^:]+)(?::\d+)?;(\d+)
    replacement: $1:$2
    target_label: __address__
  - action: labelmap
    regex: __meta_kubernetes_pod_label_(.+)
  - source_labels: [__meta_kubernetes_namespace]
    action: replace
    target_label: kubernetes_namespace
  - source_labels: [__meta_kubernetes_pod_name]
    action: replace
    target_label: kubernetes_pod_name

```

通过监控 apiserver 来监控所有对应的入口请求，增加 api-server 监控配置

```

  - job_name: 'kubernetes-apisservers'
    kubernetes_sd_configs:
      - role: endpoints
        scheme: https
        tls_config:
          ca_file: /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
          bearer_token_file: /var/run/secrets/kubernetes.io/serviceaccount/token
        relabel_configs:
          - source_labels: [__meta_kubernetes_namespace,
__meta_kubernetes_service_name, __meta_kubernetes_endpoint_port_name]
            action: keep
            regex: default;kubernetes;https
          - target_label: __address__
            replacement: kubernetes.default.svc:443

```

- 对 Ingress 和 Service 进行网络探测

创建 blackbox-exporter.yaml 进行网络探测

apiVersion: v1

kind: Service

metadata:

labels:

app: blackbox-exporter

name: blackbox-exporter

spec:

ports:

- name: blackbox

port: 9115

protocol: TCP

selector:

app: blackbox-exporter

type: ClusterIP

apiVersion: apps/v1

kind: Deployment

metadata:

labels:

app: blackbox-exporter

name: blackbox-exporter

spec:

replicas: 1

selector:

matchLabels:

app: blackbox-exporter

template:

metadata:

labels:

app: blackbox-exporter

spec:

containers:

- image: prom/blackbox-exporter

imagePullPolicy: IfNotPresent

name: blackbox-exporter

创建资源对象

kubectl -f blackbox-exporter.yaml

配置监控采集所有 service/ingress 信息，加入配置到配置文件

- job_name: 'kubernetes-services'

metrics_path: /probe

params:

```

    module: [http_2xx]
    kubernetes_sd_configs:
    - role: service
    relabel_configs:
    - source_labels:
[__meta_kubernetes_service_annotation_prometheus_io_probe]
      action: keep
      regex: true
    - source_labels: [__address__]
      target_label: __param_target
    - target_label: __address__
      replacement: blackbox-exporter.default.svc.cluster.local:9115
    - source_labels: [__param_target]
      target_label: instance
    - action: labelmap
      regex: __meta_kubernetes_service_label_(.+)
    - source_labels: [__meta_kubernetes_namespace]
      target_label: kubernetes_namespace
    - source_labels: [__meta_kubernetes_service_name]
      target_label: kubernetes_name

- job_name: 'kubernetes-ingresses'
  metrics_path: /probe
  params:
    module: [http_2xx]
    kubernetes_sd_configs:
    - role: ingress
    relabel_configs:
    - source_labels:
[__meta_kubernetes_ingress_annotation_prometheus_io_probe]
      action: keep
      regex: true
    - source_labels:
[__meta_kubernetes_ingress_scheme,__address__,__meta_kubernetes_ingres
s_path]
      regex: (.+);(.+);(.+)
      replacement: ${1}://${2}${3}
      target_label: __param_target
    - target_label: __address__
      replacement: blackbox-exporter.default.svc.cluster.local:9115
    - source_labels: [__param_target]
      target_label: instance
    - action: labelmap
      regex: __meta_kubernetes_ingress_label_(.+)
    - source_labels: [__meta_kubernetes_namespace]

```

```
target_label: kubernetes_namespace
- source_labels: [__meta_kubernetes_ingress_name]
target_label: kubernetes_name
```

▼ Grafana 可视化

Grafana 是一个通用的可视化工具。‘通用’意味着 Grafana 不仅仅适用于展示 Prometheus 下的监控数据，也同样适用于一些其他的数据可视化需求。在开始使用Grafana之前，我们首先需要明确一些 Grafana下的基本概念，以帮助用户能够快速理解Grafana。

▼ 基本概念

▪ 数据源 (Data Source)

对于Grafana而言，Prometheus这类为其提供 数据的对象均称为数据源 (Data Source) 。目前，Grafana官方提供了对：Graphite, InfluxDB, OpenTSDB, Prometheus, Elasticsearch, CloudWatch的支持。对于 Grafana管理员而言，只需要将这些对象以数据源的形式添加到Grafana中，Grafana便可以轻松的实现对这些 数据的可视化工作。

▪ 仪表盘 (Dashboard)

[官方 Dashboard 模板](#)

通过数据源定义好可视化的数据来源之后，对于用户而言最重要的事情就是实现数据的可视化。在Grafana中，我们通过Dashboard来组织和管理我们的数据可视化图表：

▪ 组织和用户

作为一个通用可视化工具，Grafana除了提供灵活的可视化定制能力以外，还提供了面向企业的组织级管理能力。在Grafana中Dashboard是属于一个**Organization (组织)**，通过Organization，可以在更大规模上使用Grafana，例如对于一个企业而言，我们可以创建多个Organization，其中**User (用户)**可以属于一个或多个不同的Organization。并且在不同的Organization下，可以为User赋予不同的权限。从而可以有效的根据企业的组织架构定义整个管理模型。

▼ 集成 Grafana

- 部署 Grafana

apiVersion: apps/v1

kind: Deployment

metadata:

name: grafana-core

namespace: kube-system

labels:

app: grafana

component: core

spec:

selector:

matchLabels:

app: grafana

replicas: 1

template:

metadata:

labels:

app: grafana

component: core

spec:

containers:

- image: grafana/grafana:6.5.3

name: grafana-core

imagePullPolicy: IfNotPresent

env:

The following env variables set up basic auth twith the default admin user and admin password.

- name: GF_AUTH_BASIC_ENABLED

value: "true"

- name: GF_AUTH_ANONYMOUS_ENABLED

value: "false"

- name: GF_AUTH_ANONYMOUS_ORG_ROLE

value: Admin

does not really work, because of template variables in exported

dashboards:

- name: GF_DASHBOARDS_JSON_ENABLED

value: "true"

readinessProbe:

httpGet:

path: /login

port: 3000

initialDelaySeconds: 30

timeoutSeconds: 1

volumeMounts:

- name: grafana-persistent-storage

```
    mountPath: /var
  volumes:
  - name: grafana-persistent-storage
    hostPath:
      path: /data/devops/grafana
      type: Directory
```

- 服务发现

```
apiVersion: v1
kind: Service
metadata:
  name: grafana
  namespace: kube-system
labels:
  app: grafana
  component: core
spec:
  type: NodePort
  ports:
  - port: 3000
    nodePort: 30011
  selector:
    app: grafana
    component: core
```

- 配置 Grafana 面板

添加 Prometheus 数据源

[下载 k8s 面板](#)，导入该面板

▼ kube-prometheus

- 替换国内镜像

```
sed -i 's/quay.io/quay.mirrors.ustc.edu.cn/g' prometheusOperator-
deployment.yaml
sed -i 's/quay.io/quay.mirrors.ustc.edu.cn/g' prometheus-prometheus.yaml
sed -i 's/quay.io/quay.mirrors.ustc.edu.cn/g' alertmanager-alertmanager.yaml
sed -i 's/quay.io/quay.mirrors.ustc.edu.cn/g' kubeStateMetrics-deployment.yaml
sed -i 's/k8s.gcr.io/lank8s.cn/g' kubeStateMetrics-deployment.yaml
sed -i 's/quay.io/quay.mirrors.ustc.edu.cn/g' nodeExporter-daemonset.yaml
sed -i 's/quay.io/quay.mirrors.ustc.edu.cn/g' prometheusAdapter-
deployment.yaml
sed -i 's/k8s.gcr.io/lank8s.cn/g' prometheusAdapter-deployment.yaml
```

```
# 查看是否还有国外镜像
grep "image: " * -r
```

- 修改访问入口

分别修改 prometheus、alertmanager、grafana 中的 spec.type 为 NodePort 方便测试访问

- 安装

```
kubectl apply --server-side -f manifests/setup
```

```
kubectl wait \
```

```
--for condition=Established \
```

```
--all CustomResourceDefinition \
```

```
--namespace=monitoring
```

```
kubectl apply -f manifests/
```

- 配置 Ingress

- # 通过域名访问（没有域名可以在主机配置 hosts）

- 192.168.113.121 grafana.wolfcode.cn

- 192.168.113.121 prometheus.wolfcode.cn

- 192.168.113.121 alertmanager.wolfcode.cn

- # 创建 prometheus-ingress.yaml

- apiVersion: networking.k8s.io/v1

- kind: Ingress

- metadata:

- namespace: monitoring

- name: prometheus-ingress

- spec:

- ingressClassName: nginx

- rules:

- host: grafana.wolfcode.cn # 访问 Grafana 域名

- http:

- paths:

- path: /

- pathType: Prefix

- backend:

- service:

- name: grafana

- port:

- number: 3000

- host: prometheus.wolfcode.cn # 访问 Prometheus 域名

- http:

- paths:

- path: /

- pathType: Prefix

- backend:

- service:

- name: prometheus-k8s

- port:

- number: 9090

- host: alertmanager.wolfcode.cn # 访问 alertmanager 域名

- http:

- paths:

- path: /

- pathType: Prefix

- backend:

- service:

- name: alertmanager-main

- port:

- number: 9093

```
# 创建 ingress
kubectl apply -f prometheus-ingress.yaml
```

- 卸载

```
kubectl delete --ignore-not-found=true -f manifests/ -f manifests/setup
```

- ▼ ELK 日志管理

- ▼ ELK 组成

- Elasticsearch

- ES 作为一个搜索型文档数据库，拥有优秀的搜索能力，以及提供了丰富的 REST API 让我们可以轻松的调用接口。

- Filebeat

- Filebeat 是一款轻量的数据收集工具

- Logstash

- 通过 Logstash 同样可以进行日志收集，但是若每一个节点都需要收集时，部署 Logstash 有点过重，因此这里主要用到 Logstash 的数据清洗能力，收集交给 Filebeat 去实现

- Kibana

- Kibana 是一款基于 ES 的可视化操作界面工具，利用 Kibana 可以实现非常方便的 ES 可视化操作

- ▼ 集成 ELK

- 部署 es 搜索服务

需要提前给 es 节点打上标签

```
kubectl label node <node name> es=data
```

创建 es.yaml

apiVersion: v1

kind: Service

metadata:

name: elasticsearch-logging

namespace: kube-logging

labels:

k8s-app: elasticsearch-logging

kubernetes.io/cluster-service: "true"

addonmanager.kubernetes.io/mode: Reconcile

kubernetes.io/name: "Elasticsearch"

spec:

ports:

- port: 9200

protocol: TCP

targetPort: db

selector:

k8s-app: elasticsearch-logging

RBAC authn and authz

apiVersion: v1

kind: ServiceAccount

metadata:

name: elasticsearch-logging

namespace: kube-logging

labels:

k8s-app: elasticsearch-logging

kubernetes.io/cluster-service: "true"

addonmanager.kubernetes.io/mode: Reconcile

kind: ClusterRole

apiVersion: rbac.authorization.k8s.io/v1

metadata:

name: elasticsearch-logging

labels:

k8s-app: elasticsearch-logging

kubernetes.io/cluster-service: "true"

addonmanager.kubernetes.io/mode: Reconcile

rules:

- apiGroups:

```

- ""
resources:
- "services"
- "namespaces"
- "endpoints"
verbs:
- "get"
---
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  namespace: kube-logging
  name: elasticsearch-logging
  labels:
    k8s-app: elasticsearch-logging
    kubernetes.io/cluster-service: "true"
    addonmanager.kubernetes.io/mode: Reconcile
subjects:
- kind: ServiceAccount
  name: elasticsearch-logging
  namespace: kube-logging
  apiGroup: ""
roleRef:
  kind: ClusterRole
  name: elasticsearch-logging
  apiGroup: ""
---
# Elasticsearch deployment itself
apiVersion: apps/v1
kind: StatefulSet #使用statefulset创建Pod
metadata:
  name: elasticsearch-logging #pod名称,使用statefulSet创建的Pod是有序号有顺序的
  namespace: kube-logging #命名空间
  labels:
    k8s-app: elasticsearch-logging
    kubernetes.io/cluster-service: "true"
    addonmanager.kubernetes.io/mode: Reconcile
    srv: srv-elasticsearch
spec:
  serviceName: elasticsearch-logging #与svc相关联, 这可以确保使用以下DNS地址访问Statefulset中的每个pod (es-cluster-[0,1,2].elasticsearch.elk.svc.cluster.local)
  replicas: 1 #副本数量,单节点
  selector:

```

```
matchLabels:
  k8s-app: elasticsearch-logging #和pod template配置的labels相匹配
template:
  metadata:
    labels:
      k8s-app: elasticsearch-logging
      kubernetes.io/cluster-service: "true"
  spec:
    serviceAccountName: elasticsearch-logging
    containers:
      - image: docker.io/library/elasticsearch:7.9.3
        name: elasticsearch-logging
        resources:
          # need more cpu upon initialization, therefore burstable class
          limits:
            cpu: 1000m
            memory: 2Gi
          requests:
            cpu: 100m
            memory: 500Mi
        ports:
          - containerPort: 9200
            name: db
            protocol: TCP
          - containerPort: 9300
            name: transport
            protocol: TCP
        volumeMounts:
          - name: elasticsearch-logging
            mountPath: /usr/share/elasticsearch/data/ #挂载点
      env:
        - name: "NAMESPACE"
          valueFrom:
            fieldRef:
              fieldPath: metadata.namespace
        - name: "discovery.type" #定义单节点类型
          value: "single-node"
        - name: ES_JAVA_OPTS #设置Java的内存参数，可以适当进行加大调整
          value: "-Xms512m -Xmx2g"
    volumes:
      - name: elasticsearch-logging
        hostPath:
          path: /data/es/
    nodeSelector: #如果需要匹配落盘节点可以添加 nodeSelect
      es: data
```

```

tolerations:
- effect: NoSchedule
  operator: Exists
# Elasticsearch requires vm.max_map_count to be at least 262144.
# If your OS already sets up this number to a higher value, feel free
# to remove this init container.
initContainers: #容器初始化前的操作
- name: elasticsearch-logging-init
  image: alpine:3.6
  command: ["/sbin/sysctl", "-w", "vm.max_map_count=262144"] #添加mmap计
数限制，太低可能造成内存不足的错误
  securityContext: #仅应用到指定的容器上，并且不会影响Volume
    privileged: true #运行特权容器
- name: increase-fd-ulimit
  image: busybox
  imagePullPolicy: IfNotPresent
  command: ["sh", "-c", "ulimit -n 65536"] #修改文件描述符最大数量
  securityContext:
    privileged: true
- name: elasticsearch-volume-init #es数据落盘初始化，加上777权限
  image: alpine:3.6
  command:
    - chmod
    - -R
    - "777"
    - /usr/share/elasticsearch/data/
  volumeMounts:
    - name: elasticsearch-logging
      mountPath: /usr/share/elasticsearch/data/

# 创建命名空间
kubectl create ns kube-logging

# 创建服务
kubectl create -f es.yaml

# 查看 pod 启用情况
kubectl get pod -n kube-logging

```

- 部署 logstash 数据清洗

创建 logstash.yaml 并部署服务

apiVersion: v1

kind: Service

metadata:

name: logstash

namespace: kube-logging

spec:

ports:

- port: 5044

targetPort: beats

selector:

type: logstash

clusterIP: None

apiVersion: apps/v1

kind: Deployment

metadata:

name: logstash

namespace: kube-logging

spec:

selector:

matchLabels:

type: logstash

template:

metadata:

labels:

type: logstash

srv: srv-logstash

spec:

containers:

- image: docker.io/kubeimages/logstash:7.9.3 #该镜像支持arm64和amd64两种架构

name: logstash

ports:

- containerPort: 5044

name: beats

command:

- logstash

- '-f'

- '/etc/logstash_c/logstash.conf'

env:

- name: "XPACK_MONITORING_ELASTICSEARCH_HOSTS"

value: "http://elasticsearch-logging:9200"

volumeMounts:

- name: config-volume
mountPath: /etc/logstash_c/
- name: config-yml-volume
mountPath: /usr/share/logstash/config/
- name: timezone
mountPath: /etc/localtime

resources: #logstash一定要加上资源限制，避免对其他业务造成资源抢占影

响

limits:

- cpu: 1000m
- memory: 2048Mi

requests:

- cpu: 512m
- memory: 512Mi

volumes:

- name: config-volume

configMap:

name: logstash-conf

items:

- key: logstash.conf
path: logstash.conf

- name: timezone

hostPath:

path: /etc/localtime

- name: config-yml-volume

configMap:

name: logstash-yml

items:

- key: logstash.yml
path: logstash.yml

apiVersion: v1

kind: ConfigMap

metadata:

name: logstash-conf

namespace: kube-logging

labels:

type: logstash

data:

logstash.conf: |-

input {

beats {

port => 5044

```

    }
  }
  filter {
    # 处理 ingress 日志
    if [kubernetes][container][name] == "nginx-ingress-controller" {
      json {
        source => "message"
        target => "ingress_log"
      }
      if [ingress_log][requesttime] {
        mutate {
          convert => ["[ingress_log][requesttime]", "float"]
        }
      }
      if [ingress_log][upstreamtime] {
        mutate {
          convert => ["[ingress_log][upstreamtime]", "float"]
        }
      }
      if [ingress_log][status] {
        mutate {
          convert => ["[ingress_log][status]", "float"]
        }
      }
      if [ingress_log][httphost] and [ingress_log][uri] {
        mutate {
          add_field => {"[ingress_log][entry]" => "%{[ingress_log][httphost]}%{[ingress_log][uri]}" }
        }
        mutate {
          split => ["[ingress_log][entry]", "/"]
        }
        if [ingress_log][entry][1] {
          mutate {
            add_field => {"[ingress_log][entrypoint]" => "%{[ingress_log][entry][0]}/%{[ingress_log][entry][1]}" }
            remove_field => "[ingress_log][entry]"
          }
        } else {
          mutate {
            add_field => {"[ingress_log][entrypoint]" => "%{[ingress_log][entry][0]}/"}
            remove_field => "[ingress_log][entry]"
          }
        }
      }
    }
  }
}

```

```

}
# 处理以srv进行开头的业务服务日志
if [kubernetes][container][name] =~ /^srv*/ {
  json {
    source => "message"
    target => "tmp"
  }
  if [kubernetes][namespace] == "kube-logging" {
    drop{}
  }
  if [tmp][level] {
    mutate{
      add_field => {"[applog][level]" => "%{[tmp][level]}"}
    }
    if [applog][level] == "debug"{
      drop{}
    }
  }
  if [tmp][msg] {
    mutate {
      add_field => {"[applog][msg]" => "%{[tmp][msg]}"}
    }
  }
  if [tmp][func] {
    mutate {
      add_field => {"[applog][func]" => "%{[tmp][func]}"}
    }
  }
  if [tmp][cost]{
    if "ms" in [tmp][cost] {
      mutate {
        split => ["[tmp][cost]","m"]
        add_field => {"[applog][cost]" => "%{[tmp][cost][0]}"}
        convert => ["[applog][cost]", "float"]
      }
    } else {
      mutate {
        add_field => {"[applog][cost]" => "%{[tmp][cost]}"}
      }
    }
  }
  if [tmp][method] {
    mutate {
      add_field => {"[applog][method]" => "%{[tmp][method]}"}
    }
  }
}

```

```

    }
    if [tmp][request_url] {
        mutate {
            add_field => {"[applog][request_url]" => "%{[tmp][request_url]}"}
        }
    }
    if [tmp][meta._id] {
        mutate {
            add_field => {"[applog][traceld]" => "%{[tmp][meta._id]}"}
        }
    }
    if [tmp][project] {
        mutate {
            add_field => {"[applog][project]" => "%{[tmp][project]}"}
        }
    }
    if [tmp][time] {
        mutate {
            add_field => {"[applog][time]" => "%{[tmp][time]}"}
        }
    }
    if [tmp][status] {
        mutate {
            add_field => {"[applog][status]" => "%{[tmp][status]}"}
            convert => ["[applog][status]", "float"]
        }
    }
}
mutate {
    rename => ["kubernetes", "k8s"]
    remove_field => "beat"
    remove_field => "tmp"
    remove_field => "[k8s][labels][app]"
}
}
output {
    elasticsearch {
        hosts => ["http://elasticsearch-logging:9200"]
        codec => json
        index => "logstash-%{+YYYY.MM.dd}" #索引名称以logstash+日志进行每日
新建
    }
}
---

```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: logstash-yml
  namespace: kube-logging
labels:
  type: logstash
data:
  logstash.yml: |-
    http.host: "0.0.0.0"
    xpack.monitoring.elasticsearch.hosts: http://elasticsearch-logging:9200
```

- 部署 filebeat 数据采集

创建 filebeat.yaml 并部署

apiVersion: v1

kind: ConfigMap

metadata:

name: filebeat-config

namespace: kube-logging

labels:

k8s-app: filebeat

data:

filebeat.yml: |-

filebeat.inputs:

- type: container

enable: true

paths:

- /var/log/containers/*.log #这里是filebeat采集挂载到pod中的日志目录

processors:

- add_kubernetes_metadata: #添加k8s的字段用于后续的数据清洗

host: \${NODE_NAME}

matchers:

- logs_path:

logs_path: "/var/log/containers/"

#output.kafka: #如果日志量较大，es中的日志有延迟，可以选择在filebeat和logstash中间加入kafka

hosts: ["kafka-log-01:9092", "kafka-log-02:9092", "kafka-log-03:9092"]

topic: 'topic-test-log'

version: 2.0.0

output.logstash: #因为还需要部署logstash进行数据的清洗，因此filebeat是把数据推到logstash中

hosts: ["logstash:5044"]

enabled: true

apiVersion: v1

kind: ServiceAccount

metadata:

name: filebeat

namespace: kube-logging

labels:

k8s-app: filebeat

apiVersion: rbac.authorization.k8s.io/v1

kind: ClusterRole

metadata:

name: filebeat

```

labels:
  k8s-app: filebeat
rules:
- apiGroups: ["" ] # "" indicates the core API group
  resources:
    - namespaces
    - pods
  verbs: ["get", "watch", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: filebeat
subjects:
- kind: ServiceAccount
  name: filebeat
  namespace: kube-logging
roleRef:
  kind: ClusterRole
  name: filebeat
  apiGroup: rbac.authorization.k8s.io
---
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: filebeat
  namespace: kube-logging
labels:
  k8s-app: filebeat
spec:
  selector:
    matchLabels:
      k8s-app: filebeat
  template:
    metadata:
      labels:
        k8s-app: filebeat
    spec:
      serviceAccountName: filebeat
      terminationGracePeriodSeconds: 30
      containers:
      - name: filebeat
        image: docker.io/kubeimages/filebeat:7.9.3 #该镜像支持arm64和amd64两种
        架构
        args: [

```

```

    "-c", "/etc/filebeat.yml",
    "-e", "-httpprof", "0.0.0.0:6060"
  ]
  #ports:
  # - containerPort: 6060
  #   hostPort: 6068
  env:
  - name: NODE_NAME
    valueFrom:
      fieldRef:
        fieldPath: spec.nodeName
  - name: ELASTICSEARCH_HOST
    value: elasticsearch-logging
  - name: ELASTICSEARCH_PORT
    value: "9200"
  securityContext:
    runAsUser: 0
    # If using Red Hat OpenShift uncomment this:
    #privileged: true
  resources:
    limits:
      memory: 1000Mi
      cpu: 1000m
    requests:
      memory: 100Mi
      cpu: 100m
  volumeMounts:
  - name: config #挂载的是filebeat的配置文件
    mountPath: /etc/filebeat.yml
    readOnly: true
    subPath: filebeat.yml
  - name: data #持久化filebeat数据到宿主机上
    mountPath: /usr/share/filebeat/data
  - name: varlibdockercontainers #这里主要是把宿主机上的源日志目录挂载到
    #filebeat容器中,如果没有修改docker或者containerd的runtime进行了标准的日志
    #落盘路径, 可以把mountPath改为/var/lib
    mountPath: /var/lib
    readOnly: true
  - name: varlog #这里主要是把宿主机上/var/log/pods和/var/log/containers的
    #软链接挂载到filebeat容器中
    mountPath: /var/log/
    readOnly: true
  - name: timezone
    mountPath: /etc/localtime
  volumes:

```

```
- name: config
  configMap:
    defaultMode: 0600
    name: filebeat-config
- name: varlibdockercontainers
  hostPath: #如果没有修改docker或者containerd的runtime进行了标准的日志
落盘路径, 可以把path改为/var/lib
    path: /var/lib
- name: varlog
  hostPath:
    path: /var/log/
  # data folder stores a registry of read status for all files, so we don't send
everything again on a Filebeat pod restart
- name: inputs
  configMap:
    defaultMode: 0600
    name: filebeat-inputs
- name: data
  hostPath:
    path: /data/filebeat-data
    type: DirectoryOrCreate
- name: timezone
  hostPath:
    path: /etc/localtime
tolerations: #加入容忍能够调度到每一个节点
- effect: NoExecute
  key: dedicated
  operator: Equal
  value: gpu
- effect: NoSchedule
  operator: Exists
```

- 部署 kibana 可视化界面

此处有配置 kibana 访问域名，如果没有域名则需要在本机配置 hosts
192.168.113.121 kibana.wolfcode.cn

创建 kibana.yaml 并创建服务

apiVersion: v1

kind: ConfigMap

metadata:

namespace: kube-logging

name: kibana-config

labels:

k8s-app: kibana

data:

kibana.yml: |-

server.name: kibana

server.host: "0"

i18n.locale: zh-CN

#设置默认语言为中文

elasticsearch:

hosts: \${ELASTICSEARCH_HOSTS} #es集群连接地址，由于我这都是k8s

部署且在一个ns下，可以直接使用service name连接

apiVersion: v1

kind: Service

metadata:

name: kibana

namespace: kube-logging

labels:

k8s-app: kibana

kubernetes.io/cluster-service: "true"

addonmanager.kubernetes.io/mode: Reconcile

kubernetes.io/name: "Kibana"

srv: srv-kibana

spec:

type: NodePort

ports:

- port: 5601

protocol: TCP

targetPort: ui

selector:

k8s-app: kibana

apiVersion: apps/v1

kind: Deployment

metadata:

```
name: kibana
namespace: kube-logging
labels:
  k8s-app: kibana
  kubernetes.io/cluster-service: "true"
  addonmanager.kubernetes.io/mode: Reconcile
  srv: srv-kibana
spec:
  replicas: 1
  selector:
    matchLabels:
      k8s-app: kibana
  template:
    metadata:
      labels:
        k8s-app: kibana
    spec:
      containers:
        - name: kibana
          image: docker.io/kubeimages/kibana:7.9.3 #该镜像支持arm64和amd64两种架构
      resources:
        # need more cpu upon initialization, therefore burstable class
        limits:
          cpu: 1000m
        requests:
          cpu: 100m
      env:
        - name: ELASTICSEARCH_HOSTS
          value: http://elasticsearch-logging:9200
      ports:
        - containerPort: 5601
          name: ui
          protocol: TCP
      volumeMounts:
        - name: config
          mountPath: /usr/share/kibana/config/kibana.yml
          readOnly: true
          subPath: kibana.yml
      volumes:
        - name: config
          configMap:
            name: kibana-config
---
apiVersion: networking.k8s.io/v1
```

```
kind: Ingress
metadata:
  name: kibana
  namespace: kube-logging
spec:
  ingressClassName: nginx
  rules:
  - host: kibana.wolfcode.cn
    http:
      paths:
      - path: /
        pathType: Prefix
      backend:
        service:
          name: kibana
          port:
            number: 5601
```

▪ Kibana 配置

进入 Kibana 界面，打开菜单中的 Stack Management 可以看到采集到的日志

避免日志越来越大，占用磁盘过多，进入 索引生命周期策略 界面点击 创建策略按钮

设置策略名称为 logstash-history-ilm-policy

关闭 热阶段

开启删除阶段，设置保留天数为 7 天

保存配置

为了方便在 discover 中查看日志，选择 索引模式 然后点击 创建索引模式 按钮

索引模式名称 里面配置 logstash-*

点击下一步

时间字段 选择 @timestamp

点击 创建索引模式 按钮

由于部署的单节点，产生副本后索引状态会变成 yellow，打开 dev tools，取消所有索引的副本数

PUT _all/_settings

```
{
  "number_of_replicas": 0
}
```

为了标准化日志中的 map 类型，以及解决链接索引生命周期策略，我们需要修改默认模板

PUT _template/logstash

```
{
  "order": 1,
  "index_patterns": [
    "logstash-*"
  ],
  "settings": {
    "index": {
      "lifecycle": {
        "name": "logstash-history-ilm-policy"
      },
      "number_of_shards": "2",
      "refresh_interval": "5s",
      "number_of_replicas": "0"
    }
  },
}
```

```
"mappings": {
  "properties": {
    "@timestamp": {
      "type": "date"
    },
    "applog": {
      "dynamic": true,
      "properties": {
        "cost": {
          "type": "float"
        },
        "func": {
          "type": "keyword"
        },
        "method": {
          "type": "keyword"
        }
      }
    },
    "k8s": {
      "dynamic": true,
      "properties": {
        "namespace": {
          "type": "keyword"
        },
        "container": {
          "dynamic": true,
          "properties": {
            "name": {
              "type": "keyword"
            }
          }
        },
        "labels": {
          "dynamic": true,
          "properties": {
            "srv": {
              "type": "keyword"
            }
          }
        }
      }
    },
    "geoip": {
      "dynamic": true,
```

```
"properties": {  
  "ip": {  
    "type": "ip"  
  },  
  "latitude": {  
    "type": "float"  
  },  
  "location": {  
    "type": "geo_point"  
  },  
  "longitude": {  
    "type": "float"  
  }  
}  
}  
}  
},  
"aliases": {}  
}
```

最后即可通过 discover 进行搜索了

- ▼ Kubernetes 可视化界面
 - ▼ Kubernetes Dashboard

- 安装

下载官方部署配置文件

wget

<https://raw.githubusercontent.com/kubernetes/dashboard/v2.7.0/aio/deploy/recommended.yaml>

修改属性

kind: Service

apiVersion: v1

metadata:

labels:

k8s-app: kubernetes-dashboard

name: kubernetes-dashboard

namespace: kubernetes-dashboard

spec:

type: NodePort #新增

ports:

- port: 443

targetPort: 8443

selector:

k8s-app: kubernetes-dashboard

创建资源

kubectl apply -f recommend.yaml

查看资源是否已经就绪

kubectl get all -n kubernetes-dashboard -o wide

访问测试

https://节点ip:端口

- 配置所有权限账号

- # 创建账号配置文件

- touch dashboard-admin.yaml

- # 配置文件

- apiVersion: v1

- kind: ServiceAccount

- metadata:

- labels:

- k8s-app: kubernetes-dashboard

- name: dashboard-admin

- namespace: kubernetes-dashboard

-

- apiVersion: rbac.authorization.k8s.io/v1

- kind: ClusterRoleBinding

- metadata:

- name: dashboard-admin-cluster-role

- roleRef:

- apiGroup: rbac.authorization.k8s.io

- kind: ClusterRole

- name: cluster-admin

- subjects:

- kind: ServiceAccount

- name: dashboard-admin

- namespace: kubernetes-dashboard

- # 创建资源

- kubectl apply -f dashboard-admin.yaml

- # 查看账号信息

- kubectl describe serviceaccount dashboard-admin -n kubernetes-dashboard

- # 获取账号的 token 登录 dashboard

- kubectl describe secrets dashboard-admin-token-5crbd -n kubernetes-dashboard

- Dashboard 的使用

- ▼ kubesphere

- <https://kubesphere.io/zh/>

KubeSphere 愿景是打造一个以 Kubernetes 为内核的云原生分布式操作系统，它的架构可以非常方便地使第三方应用与云原生生态组件进行即插即用（plug-and-play）的集成，支持云原生应用在多云与多集群的统一分发和运维管理。

- 本地存储动态 PVC

- # 在所有节点安装 iSCSI 协议客户端 (OpenEBS 需要该协议提供存储支持)

- yum install iscsi-initiator-utils -y

- # 设置开机启动

- systemctl enable --now iscsid

- # 启动服务

- systemctl start iscsid

- # 查看服务状态

- systemctl status iscsid

- # 安装 OpenEBS

- kubectl apply -f <https://openebs.github.io/charts/openebs-operator.yaml>

- # 查看状态 (下载镜像可能需要一些时间)

- kubectl get all -n openebs

- # 在主节点创建本地 storage class

- kubectl apply -f default-storage-class.yaml

- 安装

- # 安装资源

- kubectl apply -f <https://github.com/kubesphere/ks-installer/releases/download/v3.3.1/kubesphere-installer.yaml>

- kubectl apply -f <https://github.com/kubesphere/ks-installer/releases/download/v3.3.1/cluster-configuration.yaml>

- # 检查安装日志

- kubectl logs -n kubesphere-system \$(kubectl get pod -n kubesphere-system -l 'app in (ks-install, ks-installer)' -o jsonpath='{.items[0].metadata.name}') -f

- # 查看端口

- kubectl get svc/ks-console -n kubesphere-system

- # 默认端口是 30880, 如果是云服务商, 或开启了防火墙, 记得要开放该端口

- # 登录控制台访问, 账号密码: admin/P@88w0rd

- 启用可插拔组件

- <https://kubesphere.io/zh/docs/v3.3/pluggable-components/>

- Rancher

- <https://www.rancher.cn/>

- Kuboard

<https://www.kuboard.cn/>

▼ ⑤ V. Spring Cloud Alibaba 微服务 DevOps 实战

▼ DevOps 环境搭建

▼ Gitlab

GitLab 是一个用于仓库管理系统的开源项目，使用 Git 作为代码管理工具，并在此基础上搭建起来的 Web 服务。

Gitlab 是被广泛使用的基于 git 的开源代码管理平台，基于 Ruby on Rails 构建，主要针对软件开发过程中产生的代码和文档进行管理，Gitlab 主要针对 group 和 project 两个维度进行代码和文档管理，其中 group 是群组，project 是工程项目，一个 group 可以管理多个 project，可以理解为一个群组中有多项软件开发任务，而一个 project 中可能包含多个 branch，意为每个项目中有多个分支，分支间相互独立，不同分支可以进行归并。

▪ 安装 Gitlab

下载安装包

wget https://mirrors.tuna.tsinghua.edu.cn/gitlab-ce/yum/el7/gitlab-ce-15.9.1-ce.0.el7.x86_64.rpm

安装

rpm -i gitlab-ce-15.9.1-ce.0.el7.x86_64.rpm

编辑 /etc/gitlab/gitlab.rb 文件

修改 external_url 访问路径 http://<ip>:<port>

其他配置修改如下

gitlab_rails['time_zone'] = 'Asia/Shanghai'

puma['worker_processes'] = 2

sidekiq['max_concurrency'] = 8

postgresql['shared_buffers'] = "128MB"

postgresql['max_worker_processes'] = 4

prometheus_monitoring['enable'] = false

更新配置并重启

gitlab-ctl reconfigure

gitlab-ctl restart

- 页面配置

- # 查看默认密码

- cat /etc/gitlab/initial_root_password

- # 登录后修改默认密码 > 右上角头像 > Preferences > Password

- # 修改系统配置：点击左上角三横 > Admin

- # Settings > General > Account and limit > 取消 Gravatar enabled > Save changes

- # 关闭用户注册功能

- # Settings > General > Sign-up restrictions > 取消 Sign-up enabled > Save changes

- # 开启 webhook 外部访问

- # Settings > Network > Outbound requests > Allow requests to the local network from web hooks and services 勾选

- # 设置语言为中文（全局）

- # Settings > Preferences > Localization > Default language > 选择简体中文 > Save changes

- # 设置当前用户语言为中文

- # 右上角用户头像 > Preferences > Localization > Language > 选择简体中文 > Save changes

- 配置 Secret

- # 创建 gitlab 默认用户名密码 secret

- echo root > ./username

- echo wolcode > password

- kubectl create secret generic git-user-pass --from-file=./username --from-file=./password -n kube-devops

- 为项目配置 Webhook

- 进入项目点击侧边栏设置 > Webhooks 进入配置即可

- URL：在 jenkins 创建 pipeline 项目后

- 触发来源：

- 推送事件：表示收到新的推送代码就会触发

- 标签推送事件：新标签推送才会触发

- 评论：根据评论决定触发

- 合并请求事件：创建、更新或合并请求触发

- 添加成功后，可以在下方点击测试按钮查看 jenkins 是否成功触发构建操作

- 卸载

- # 停止服务

- gitlab-ctl stop

- # 卸载 rpm 软件（注意安装的软件版本是 ce 还是 ee）

- rpm -e gitlab-ce

- # 查看进程

- ps -ef|grep gitlab

- # 干掉第一个 runsvdir -P /opt/gitlab/service log 进程

- # 删除 gitlab 残余文件

- find / -name *gitlab* | xargs rm -rf

- find / -name gitlab | xargs rm -rf

- ▼ Harbor

- 安装 Harbor

- # 下载 harbor 安装包

- # 解压后执行 [install.sh](#) 就行

- 配置 Secret

- # 创建 harbor 访问账号密码（需要将下访问的配置信息改成你自己的）

- kubectl create secret docker-registry harbor-secret --docker-server=192.168.113.122:8858 --docker-username=admin --docker-password=wolfcode -n kube-devops

- ▼ SonarQube

- 安装 SonarQube

- # 进入 /opt/k8s/devops

- kubectl apply -f sonarqube/

- 生成服务 token

- # 登录到 sonarqube 后台，点击头像 > MyAccount > Security > Generate Tokens > generate 生成 token 并复制

- 创建 Webhook 服务

- # 点击顶部菜单栏的配置 > 配置（小三角）> 网络调用

- Name: wolfcode-jenkins

- URL: http://<sonar ip>:<sonar port>/sonarqube-webhook/

- 创建项目

SonarQube 顶部菜单栏 Projects > Create new project > 配置基础信息并保存 > Provide a token > Generate 生成 token > Continue

分别选择 Java / Maven 后, 按照脚本配置 Jenkinsfile 中的 sonar 配置信息
mvn sonar:sonar -Dsonar.projectKey=k8s-cicd-demo

- ▼ Jenkins

- 构建带 maven 环境的 jenkins 镜像

构建带 maven 环境的 jenkins 镜像

docker build -t 192.168.113.122:8858/library/jenkins-maven:jdk-11.

登录 harbor

docker login -uadmin 192.168.113.122:8858

推送镜像到 harbor

docker push 192.168.113.122:8858/library/jenkins-maven:jdk-11

- 安装 Jenkins

进入 jenkins 目录, 安装 jenkins

kubectl apply -f manifests/

查看是否运行成功

kubectl get po -n kube-devops

查看 service 端口, 通过浏览器访问

kubectl get svc -n kube-devops

查看容器日志, 获取默认密码

kubectl logs -f pod名称 -n kube-devops

- ▼ 安装插件

如果有些插件搜不到, 可以去官网下载 <https://plugins.jenkins.io/>

- Build Authorization Token Root

构建授权 token

- Gitlab

gitlab 配置插件

- **SonarQube Scanner**

代码质量审查工具

在 Dashboard > 系统管理 > Configure System 下面配置 SonarQube servers

Name: sonarqube # 注意这个名字要在 Jenkinsfile 中用到

Server URL: http://sonarqube:9000

Server authentication token: 创建 credentials 配置为从 sonarqube 中得到的 token

进入系统管理 > 全局工具配置 > SonarQube Scanner > Add SonarQube Scanner

Name: sonarqube-scanner

自动安装: 取消勾选

SONAR_RUNNER_HOME: /usr/local/sonar-scanner-cli

- **Node and Label parameter**

节点标签参数配置

- **Kubernetes**

jenkins + k8s 环境配置

进入 Dashboard > 系统管理 > 节点管理 > Configure Clouds 页面

配置 k8s 集群

名称: kubernetes

点击 Kubernetes Cloud details 继续配置

Kubernetes 地址:

如果 jenkins 是运行在 k8s 容器中, 直接配置服务名即可

https://kubernetes.default

如果 jenkins 部署在外部, 那么则不仅要配置外部访问 ip 以及 apiserver 的端口 (6443), 还需要配置服务证书

Jenkins 地址:

如果部署在 k8s 集群内部: http://jenkins-service.kube-devops

如果在外部: http://192.168.113.120:32479 (换成你们自己的)

配置完成后保存即可

- **Config File Provider**

用于加载外部配置文件, 如 Maven 的 settings.xml 或者 k8s 的 kubeconfig 等

- **Git Parameter**

git 参数插件, 在进行项目参数化构建时使用

- 创建 gitlab 访问凭证

系统管理 > 安全 > Manage Credentials > System > 全局凭据 (unrestricted) > Add Credentials

范围：全局

用户名：root

密码：wolfcode

ID：gitlab-user-pass

- ▼ 案例：SpringBoot 项目 CICD

点击创建任务 > Pipeline (流水线)

参数化构建：可以通过配置参数化构建，将经常变动的参数提取出来，方便每次构建时传入

构建触发器：勾选基于 gitlab webhook 的构建方式，同时拿到 webhook 地址，且在高级中生成 secret token，将其配置到 gitlab 项目中

根据需求选择 webhook 的触发事件

流水线：选择定义为 Pipeline script from SCM 从远程仓库拉取 Jenkinsfile 配置
配置 SCM 为 Git

Repositories：

Repository URL：仓库地址

Credentials：仓库访问的账号密码

Branches to build：选择拉取哪个分支下的代码

脚本路径：Jenkinsfile 脚本文件名称以及所在路径

PS：当项目下载下来后，所有操作都默认在项目根目录下执行

- 配置节点标签

系统管理 > 节点管理 > 列表中 master 节点最右侧的齿轮按钮

修改标签的值与项目中 Jenkinsfile 中 agent > kubernetes > label 的值相匹配

- 创建流水线项目

在首页点击 Create a Job 创建一个流水线风格的项目

- Webhook 构建触发器

----- Jenkins 流水线项目 Webhook 配置 -----
在 Jenkins 项目配置下找到构建触发器栏目

勾选 Build when a change is pushed to GitLab. GitLab webhook URL:
<http://192.168.113.121:31216/project/k8s-cicd-demo>

上方的 URL 就是用于配置到 gitlab 项目 webhook 的地址

启用 Gitlab 构建触发器:

Push Events: 勾选, 表示有任意推送到 git 仓库的操作都会触发构建
Open Merge Request Events: 勾选, 表示有请求合并时触发构建

点击高级 > Secret Token > Generate 按钮, 生成 token

保存以上配置

----- GitLab 项目 Webhook 配置 -----
进入 GitLab 项目设置界面 > Webhooks

将上方 Jenkins 中的 URL 配置到 URL 处

将上方生成的 Secret Token 配置到 Secret 令牌

按照需求勾选触发来源, 这里我依然勾选 推送事件、合并请求事件

取消 SSL 验证

点击添加 webhook 按钮, 添加后可以点击测试确认链接是否可以访问

- Pipeline 脚本配置

流水线: 选择定义为 Pipeline script from SCM 从远程仓库拉取 Jenkinsfile 配置
配置 SCM 为 Git

Repositories:

Repository URL: 仓库地址

Credentials: 仓库访问的账号密码

Branches to build: 选择拉取哪个分支下的代码

脚本路径: Jenkinsfile 脚本文件名称以及所在路径

- ▼ 检查/创建相关凭证

- Harbor 镜像仓库凭证

通过系统管理 > Manage Credentials > 凭据 > System > 全局凭证 > Add Credentials 添加 Username with password 类型凭证

填写好用户名密码后，需要注意凭证 id 要与 Jenkinsfile 中的 **DOCKER_CREDENTIAL_ID** 一致

- Gitlab 访问凭证

通过系统管理 > Manage Credentials > 凭据 > System > 全局凭证 > Add Credentials 添加 Username with password 类型凭证

填写好用户名密码后，需要注意凭证 id 要与 Jenkinsfile 中的 **GIT_CREDENTIAL_ID** 一致

- kubeconfig 文件 id

1. 事先安装 Config File Provider 插件

2. 进入系统管理 > Mapped files > Add a new Config 添加配置文件

2.1 Type 选择 Custom file 点击 next

2.2 在 k8s master 节点执行 cat ~/.kube/config 查看文件内容，并将所有内容复制

2.3 将复制的内容贴到 Config file 的 Content 中后点击 Submit 保存并提交

3. 复制保存后文件 id 到 Jenkinsfile 中的 **KUBECONFIG_CREDENTIAL_ID** 处

- SonarQube 凭证

1. 进入 SonarQube 系统，点击右上角用户头像 > 我的账号 进入设置页面

2. 点击 安全 > 填写令牌名称 > 点击生成按钮生成 token > 复制生成后的 token

3. 进入 jenkins 添加凭证管理页面，添加 Secret Text 类型的凭证，将 token 贴入其中

4. 保证凭证 id 与 Jenkinsfile 文件中的 **SONAR_CREDENTIAL_ID** 一致

- 添加 SonarQube Webhook

1. 进入 SonarQube 管理页面，点击顶部菜单栏的配置 > 配置(小三角) > 网络调用

2. 点击右侧创建按钮创建新的 Webhook，并填写名称与地址

名称: jenkins

地址: http://jenkins访问ip:端口/sonarqube-webhook/

- 项目构建

方式一：在 Jenkins 管理后台，进入项目中点击立即构建进行项目构建

方式二：在开发工具中修改代码，并将代码提交到远程仓库自动触发构建

- ▼ 微服务 DevOps 实战

- ▼ 项目构建

▼ 项目环境

- MySQL
- Nacos
- Redis
- MongoDB
- Elasticsearch
- RocketMQ

▼ 服务

- API 网关
- 用户服务
- 商品服务
- 秒杀服务
- 前端服务

▼ Jenkins CI/CD

- 创建流水线项目
- Extended Choice Parameter

先安装插件，然后创建 Job 进入配置页面中找到参数化构建下添加参数自定义下拉框选项参数，用于配置需要部署的项目信息

Name: PROJECT_NAME

Description: 请选择需要构建的服务

Parameter Type: Check Boxes

Number of Visible Items: 4

Delimiter: ,

Value: shop-ui@80,api-gateway@9000,shop-uaa@8031,shop-product@8041,shop-flashsale@8061

Default Value: shop-ui@80

Description: 前端页面,服务网关,用户服务,商品服务,秒杀服务

▼ Kubesphere DevOps

- 开启 DevOps
- 集成 SonarQube

- 更新 settings.xml

在集群管理 > 配置 > 配置字典下搜索 ks-devops-agent
点进去后点击更多操作 > 编辑 YAML 添加如下内容并保存

```
<servers>
  <server>
    <id>releases</id>
    <username>admin</username>
    <password>wolfcode</password>
  </server>
  <server>
    <id>snapshots</id>
    <username>admin</username>
    <password>wolfcode</password>
  </server>
</servers>

<mirrors>
  <mirror>
    <id>releases</id>
    <name>nexus maven</name>
    <mirrorOf>*</mirrorOf>
    <url>http://192.168.113.121:8868/repository/maven-public/</url>
  </mirror>
</mirrors>
```

- ▼ 部署项目

- ▼ Spring Boot 项目

- ▼ 项目

- ks-cicd-demo

- ▼ 配置文件

- harbor-secret

- ▼ 微服务项目

- ▼ 项目

- ks-shop-dev
 - ks-shop-flashsale

- ▼ 配置文件

- harbor-secret

▼ DevOps 项目

▼ Spring Boot 项目

▼ 流水线

- cicd-demo

▼ 配置

- gitlab-user-pass
- harbor-user-pass
- kubeconfig-id

▼ 构建参数

- 分支
字符串参数
- 版本号
字符串参数

▼ 微服务项目

▼ 流水线

- flashsale-cicd

▼ 配置

- gitlab-user-pass
- harbor-user-pass
- kubeconfig-id

▼ 构建参数

▼ 服务

多选参数

- frontend-server
- shop-parent/api-gateway
- shop-parent/shop-uaa
- shop-parent/shop-provider/flashsale-server
- shop-parent/shop-provider/product-server

▼ 命名空间

多选参数

- snapshots
- releases

▼ 副本数

多选参数

- 1
- 3
- 5
- 7

▪ 版本号

字符串参数

▪ 分支

字符串参数